

Chapter 11

Multiway Search Trees

11.1 m-Way Search trees

- ✓ If an AVL tree is used to represent a **very large collection of elements**, it would be stored on **disk**.
- ✓ Thus, to search among n keys may requires **h disk accesses** if each node lies in a **different block** in the worst case.
- ✓ If $n=10^6$, $h \approx 1.44\log_2 (n+1) \approx 28$ --- too bad!
- ✓ If we use the **search trees whose degree is more than 2**, the tree will be shorter than $\log_2(n+1)$.

- ✓ In practice, we adopt the **maximum degree that allows the tree node to fit within a single block.**

Definition of m-way search tree

Definition: An m-way search tree, is either **empty** or satisfies the following properties:

(1) The **root** has **at most** m subtrees and has the following structure:

$$n, A_0, (E_1, A_1), (E_2, A_2), \dots, (E_n, A_n)$$

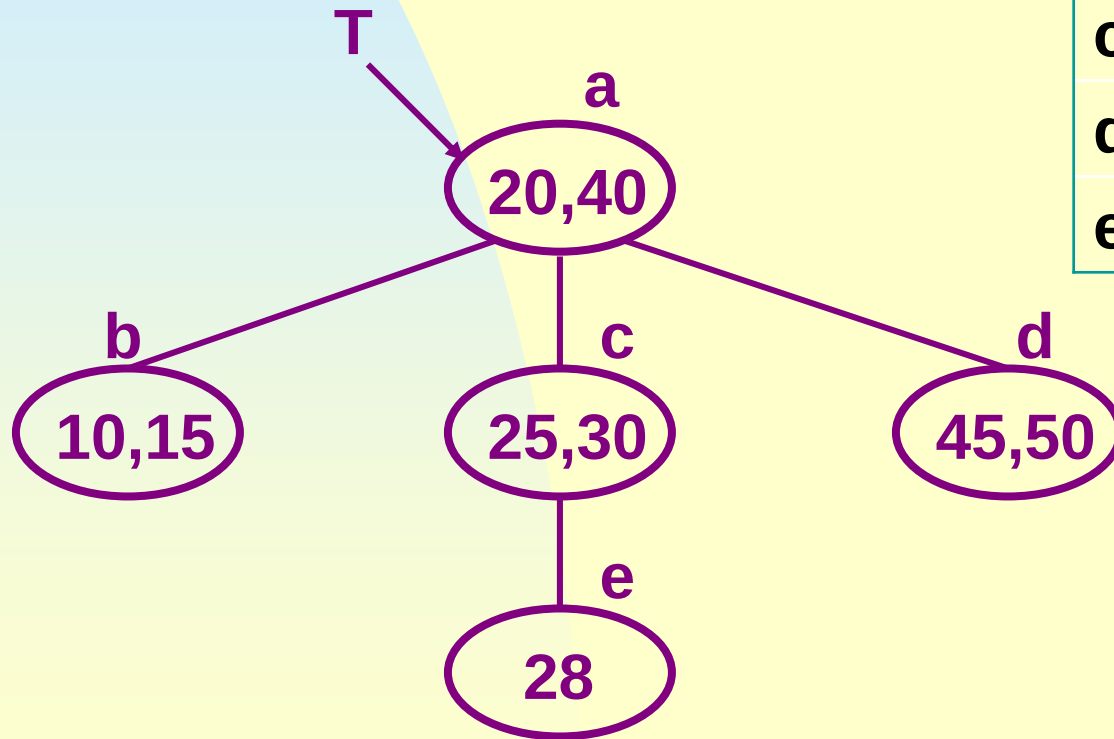
where the A_i ($0 \leq i \leq n < m$) are **pointers** to subtrees, and E_i ($1 \leq i \leq n < m$) are **elements**. Each E_i has a key $E_i.K$.

(2) $E_i.K < E_{i+1}.K$ for $1 \leq i < n$.

(3) If we assume $E_0.K = -\infty$ and $E_{n+1}.K = \infty$, then **all keys in subtree A_i are greater than $E_i.K$ and less than $E_{i+1}.K$ for $0 \leq i \leq n$.**

(4) The subtrees A_i ($0 \leq i \leq n$) are also m-way search trees.

Example: A 3-way search tree:



node	Schematic format
a	2,b,(20,c),(40,d)
b	2,0,(10,0),(15,0)
c	2,0,(25,e),(30,0)
d	2,0,(45,0),(50,0)
e	1,0,(28,0)

Some observations

- ✓ In a tree of degree m and height h , the **maximum number of nodes** is

$$\sum_{0 \leq i \leq h-1} m^i = (m^h - 1) / (m - 1)$$

- ✓ Since, each node has at most **$m-1$** keys, the **maximum number of keys** is **m^h-1** .
 - for a **binary tree** with $h=3$, it is 7.
 - for a **200-way tree** with $h=3$, it is $8 \cdot 10^6 - 1$.
- ✓ To achieve a performance close to that of the **best m -way search tree**, the search tree must be **balanced**.
- ✓ The two primary types of balanced m -way search trees are **B-trees** and **B+-trees**.

11.1.2 Searching an m-way Search Tree

Searching an m-way search tree is **similar to searching a binary search tree**. A brief algorithm description is as follows.

// Search an m-way search tree for an element with key x.

// Return the element if found, else return **NULL**.

$E_0.K = -MAXKEY$;

for (*p=root; p; p= A_i)

{

Let node p have the format “ $n, A_0, (E_1, A_1), \dots, (E_n, A_n)$ ”;

$E_{n+1}.K = MAXKEY$; //n may change with different nodes

Determine i such that $E_i.K \leq x < E_{i+1}.K$;

if ($x == E_i.K$) return E_i ;

}

return **NULL**; // x is not in the tree

11.2 B-Trees

11.2.1 Definition and Properties

- ✓ To define a B-tree, we extend the m-way search tree by introducing **external nodes**.
- ✓ An **external (or failure)** node is inserted in place of every **NULL** pointer.
- ✓ It means that **an external node** is a node that can only be accessed during a search when the target element is not in the tree.
- ✓ Nodes that are not external nodes are called **internal nodes**.

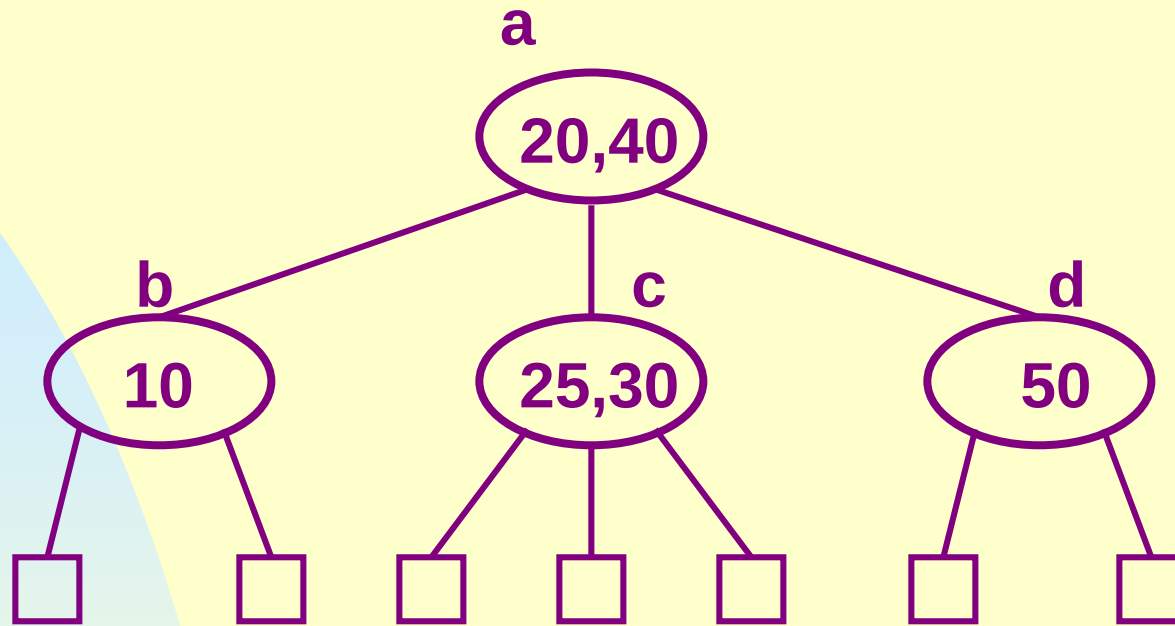
Definition: A B-tree of order m is an m -way search tree that is empty or satisfies the following properties:

(1) The root node must have at least two children—even if both of these children are external nodes.

(2) All nodes except the root node and external nodes must have at least $\lceil m/2 \rceil$ children.

(3) All external nodes must be at the same level.

Note that a B-tree of order 2 must be a full binary search tree, in accordance with the formal definition of B-trees.



A B-tree of order 3

When $m=3$, all internal nodes have a degree of either 2 or 3, so a B-tree of order 3 is also called an **2-3 tree.**

Some Explanations

- For **any $n \geq 0$ and $m > 2$** , there is always a B-tree of order m that contains n elements.
- External nodes are introduced only to simplify **definition and explanation of B-trees**. They do not have a physical representation in a computer. Actually, each external node is represented by a **NULL pointer**.

11.2.2 Number of Elements in a B-tree

Let t be a B-tree of order m where all external nodes are at level $l+1$ ($l \geq 1$), and N be the number of keys in t . Then

(1) $N \leq m^l - 1$ (upper bound).

(2) $N \geq 2 \lceil m/2 \rceil^{l-1} - 1$ (lower bound).

To show this, we can observe:

There are at least 1 internal nodes at level 1.

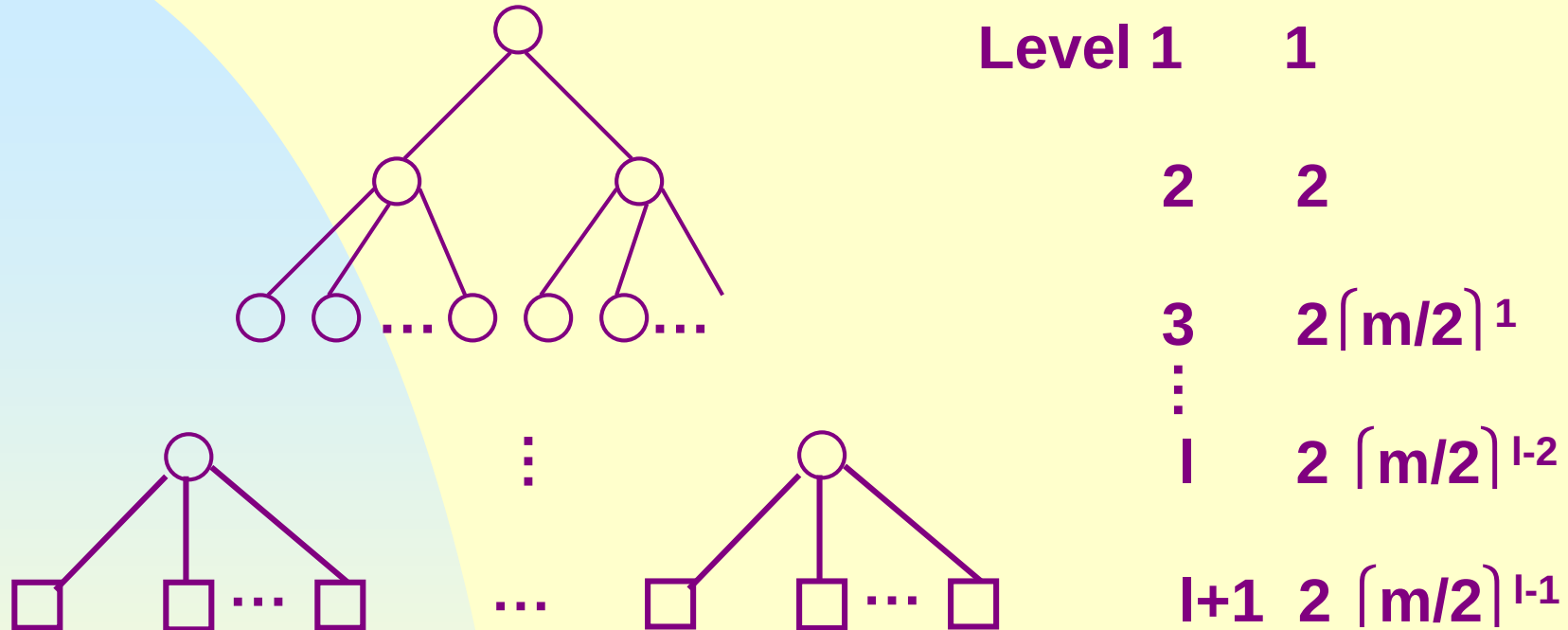
There are at least 2 internal nodes at level 2.

There are at least $2 \lceil m/2 \rceil$ internal nodes at level 3,

...

There are at least $2 \lceil m/2 \rceil^{l-2}$ internal nodes at level l .

There are at least $2 \lceil m/2 \rceil^{l-1}$ external nodes at level $l+1$.



If the key values in the tree are $K_1, K_2, \dots, K_N$, and $K_i < K_{i+1}$, $1 \leq i < N$, then the number of external nodes is $N+1$, because failures occur for $K_i < x < K_{i+1}$ ($0 \leq i \leq N$), where we let $K_0 = -\text{MAXVALUE}$, $K_{N+1} = \text{MAXVALUE}$.

Therefore,

$N+1$ = number of external (failure) nodes

= number of nodes at level $l+1$

$\geq 2 \lceil m/2 \rceil^{l-1}$

So, $N \geq 2 \lceil m/2 \rceil^{l-1} - 1$.

This implies that:

$$l \leq \log_{\lceil m/2 \rceil} ((N+1)/2) + 1.$$

If there are N keys in a B-tree of order m and a B-tree node can be retrieved with a single disk access, then the maximum number of disk accesses required for a search operation is l .

Example

Assume $m=200$ (which is quite practice for a disk resident B-tree), and note that **l is integer**:

(1) For $N \leq 2 \cdot 10^6 - 2$,

$$l \leq \log_{100}(10^6 - 1/2) + 1 < 4, \text{ so } l \leq 3.$$

(2) For $N \leq 2 \cdot 10^6 - 1$,

$$l \leq \log_{100}(10^6) + 1 = 4.$$

(3) For $N \leq 2 \cdot 10^8 - 2$,

$$l \leq \log_{100}(10^8 - 1/2) + 1 < 5, \text{ so } l \leq 4.$$

(4) For $N \leq 2 \cdot 10^{10} - 2$,

$$l \leq \log_{100}(10^{10} - 1/2) + 1 < 6, \text{ so } l \leq 5.$$

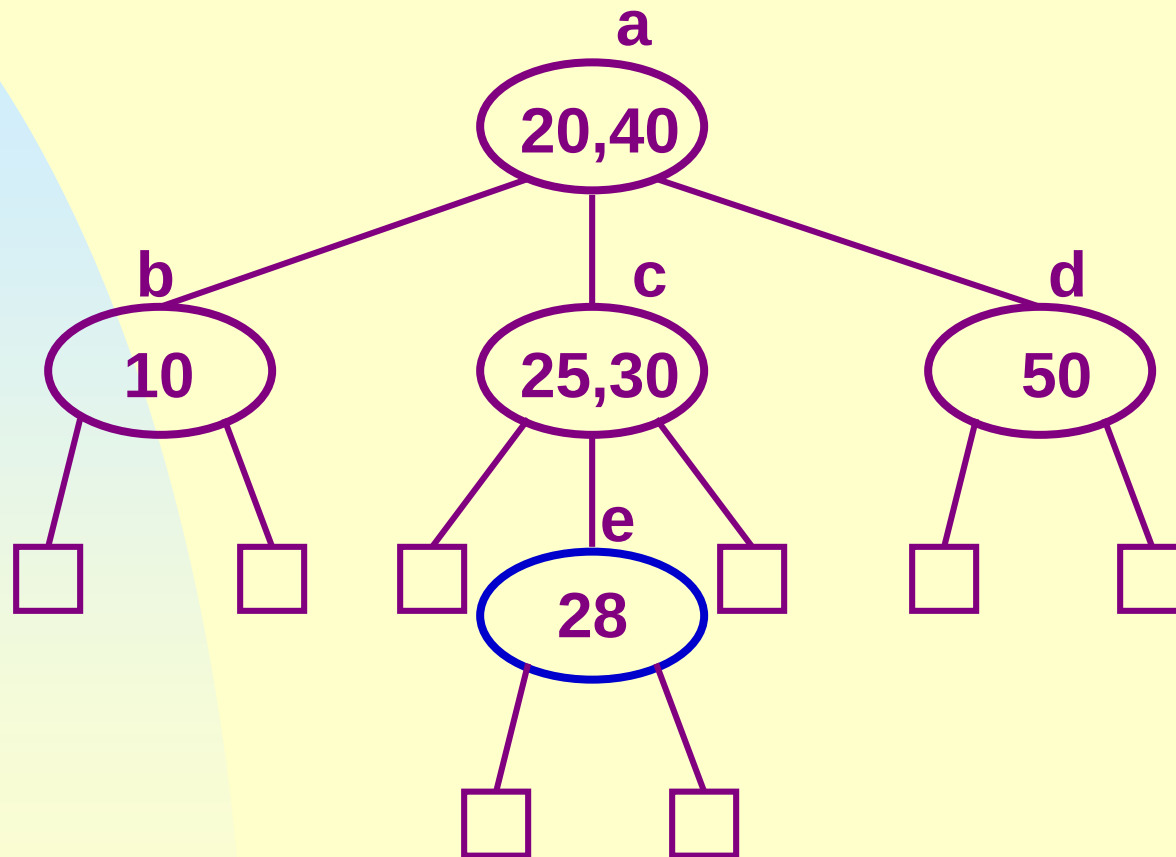
11.2.3 Insertion into a B-Tree

Note that we **can't** insert a new element in the position of an **external node** to become a new **node**, because:

- (1) One key might be too few for a node;
- (2) External nodes **must be at the same level**.

As shown in the next slide:

We **can't** insert 28 into a B-Tree like this:



■ Basic idea of the **insertion algorithm** for B-tree of order m :

- (1) Perform a search to determine the target **leaf node (not the external node)**, p , for inserting the new key.
- (2) If the insertion causes node p to contain **m keys** (exceeding its maximum capacity), p shall be split. Otherwise, the updated p is written to the disk and the insertion is completed.

How to split a node with m keys?

We assume that following the insertion of the new element, **p** has the format

m, A_0 , (E_1, A_1) , ..., (E_m, A_m) and $E_i.K < E_{i+1}.K$, $1 \leq i < m$.

The node is split into 2 nodes, p and q, with the following formats:

node **p**: $\lceil m/2 \rceil - 1$, A_0 , (E_1, A_1) , ..., $(E_{\lceil m/2 \rceil - 1}, A_{\lceil m/2 \rceil - 1})$ (11.5)

node **q**: $m - \lceil m/2 \rceil$, $A_{\lceil m/2 \rceil}$, $(E_{\lceil m/2 \rceil + 1}, A_{\lceil m/2 \rceil + 1})$, ..., (E_m, A_m)

Meanwhile, the remaining element $E_{\lceil m/2 \rceil}$ and a new pointer to the new node, q, form a tuple $(E_{\lceil m/2 \rceil}, q)$. Then it is inserted into the **parent of p**.

Note:

- Inserting the tuple into the parent node may lead to the **split of the parent**, and the splitting process may propagate all the way up to the root.
- When the **root split**, a new root with a **single key** is generated, and the height of the B-tree increases by one.

The next slide gives a brief description of the insertion algorithm for a disk resident B-tree.

A brief description of the insertion algorithm

Program 11.2: insertion into a B-tree

//Insert element x into a disk resident B-tree.

Search the B-tree for an element E with key x.K;

if such an E is found, replace E with x and **return**;

else let p be the leaf into which x is to be inserted;

q = **NULL**;

for (e=x; p; p=p→parent()) // the parent of root is **NULL**

{ //(e, q) is to be inserted into p

Insert (e, q) into appropriate position in node p;

Let the resulting node have the form:

$$n, A_0, (E_1, A_1), \dots, (E_n, A_n);$$

if ($n \leq m-1$) { // the resulting node is not too big

write node p to disk; **return**;

}

//else node p has to be split

Let node p and q be defined as in (11.5);

$e = E_{\lceil m/2 \rceil}$;

write nodes p and q to the disk;

}

// a new root is to be created

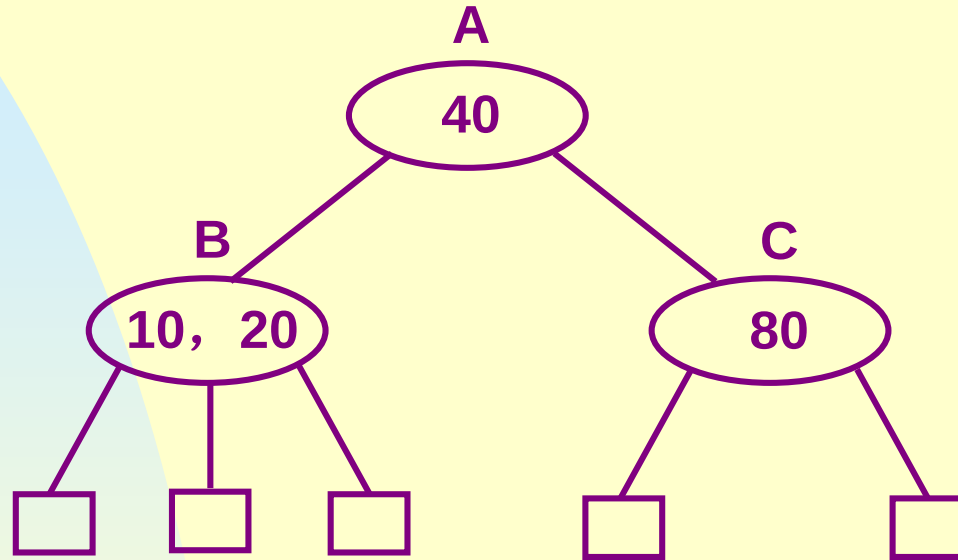
Create a new node r with format: 1, root, (e, q);

root=r;

write root to disk;

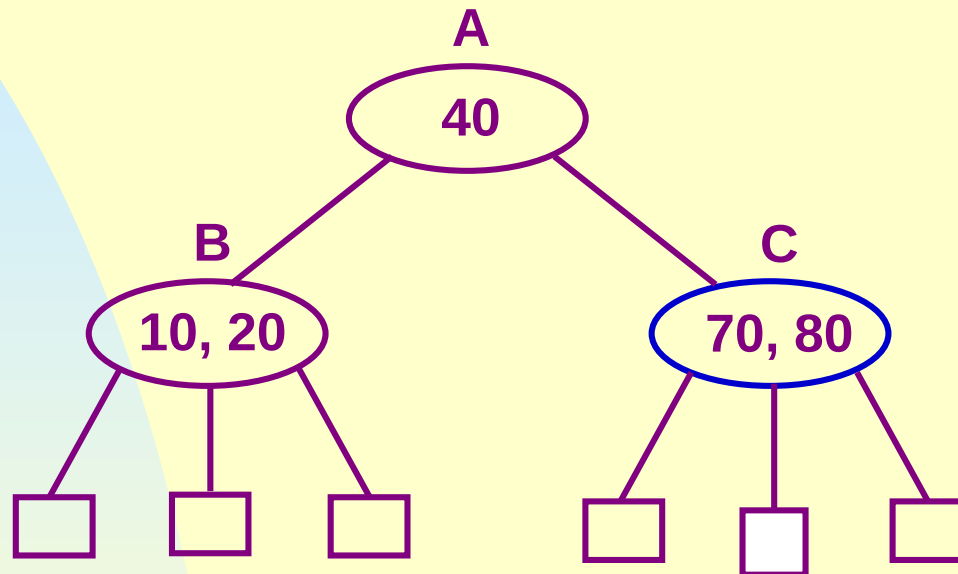
node p: $\lceil m/2 \rceil - 1, A_0, (E_1, A_1), \dots, (E_{\lceil m/2 \rceil - 1}, A_{\lceil m/2 \rceil - 1})$
node q: $m - \lceil m/2 \rceil, A_{\lceil m/2 \rceil}, (E_{\lceil m/2 \rceil + 1}, A_{\lceil m/2 \rceil + 1}), \dots, (E_m, A_m)$

Example 11.1:



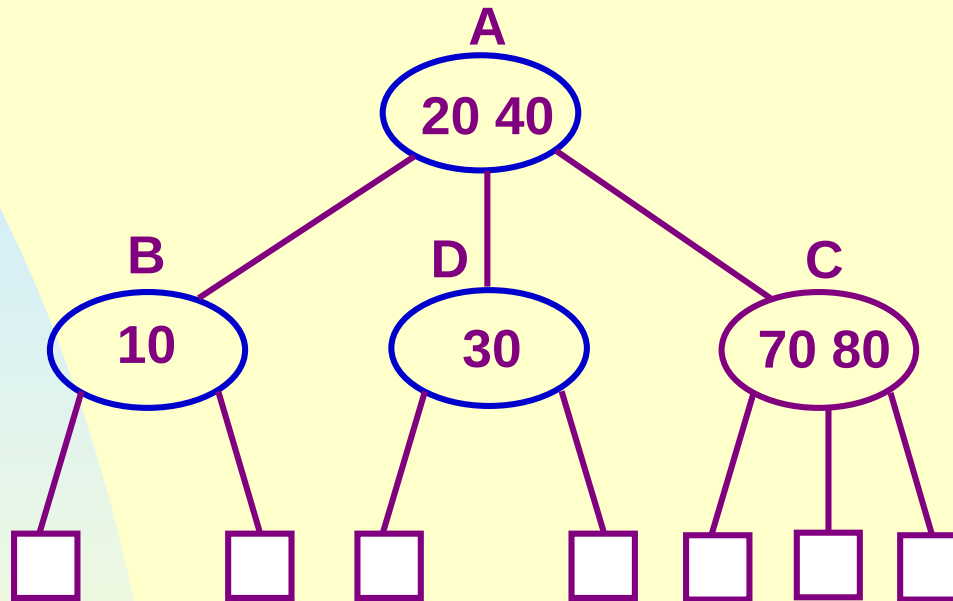
An initial 2-3 tree

Insertion order: 70, 30, 60.



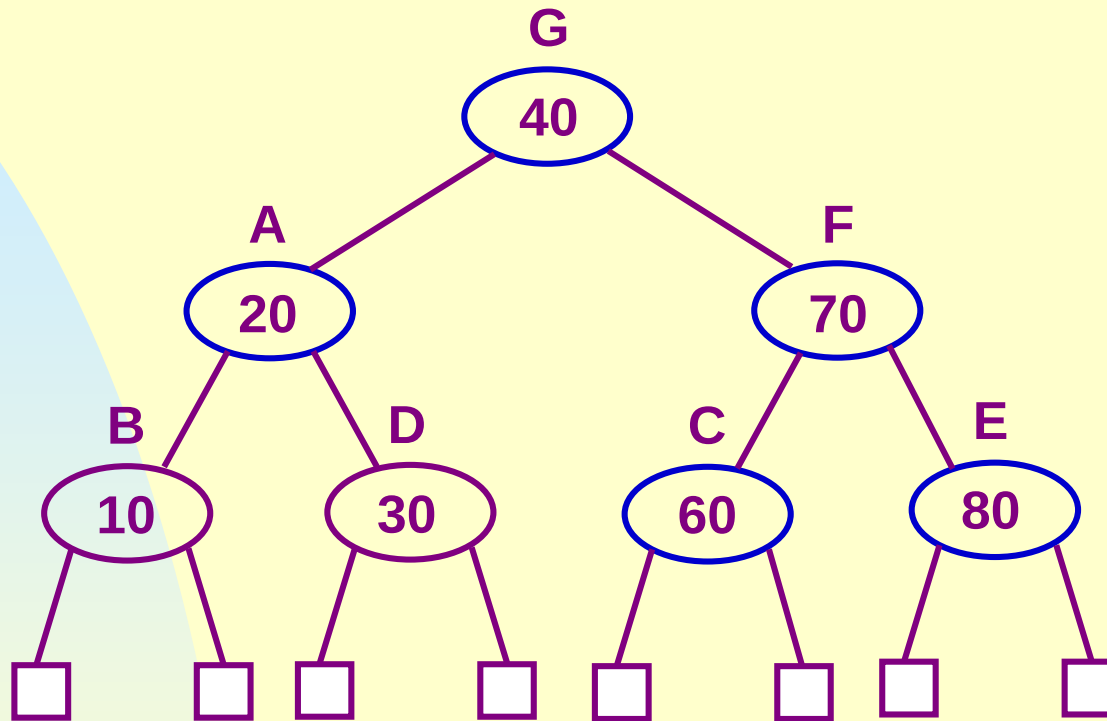
70 inserted

Insertion order: 70, 30, 60.



30 inserted

Insertion order: 70, 30, 60.



60 inserted

Insertion order: 70, 30, 60.

Analysis of B-tree Insertion:

- ✓ Let h be the height of the B-tree, then h disk accesses are made during the top-down search.
- ✓ In the worst, all h of the accessed nodes may split during the bottom-up splitting pass. When a non-root node split, 2 nodes are written out. When the root split, 3 nodes are written out.
- ✓ If we assume that the h nodes read in during the top-down pass can be stored in memory so that they are not to be retrieved from disk during the bottom-up pass, then the number of disk accesses for an insertion is at most:

$$h + 2(h-1) + 3 = 3h+1.$$

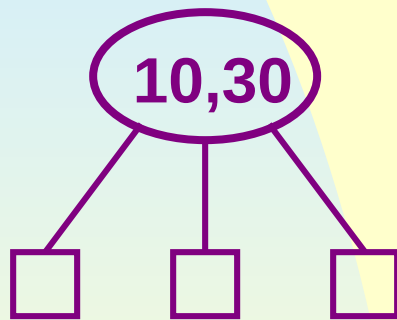


- ✓ However, the **average** number of disk accesses is approximately **$h+1$** for large m .
- ✓ To show this, suppose we start with an **empty** B-tree and insert N elements into it.
- ✓ Let **p ($p > 2$)** be the number of **internal nodes** in the final B-tree with N elements, then the total number of nodes split is **at most $p-2$** . Because:
 - Each time **a non-root node splits**, at least **one additional** node is created.
 - When the root splits, **two additional nodes** are created.
 - The **first node** is created without any splitting, and if a B-tree contains more than one node, then **the root must have split at least once**.

An example: $p-2$ is the tightest upper bound

The following example shows that $p-2$ is the tightest upper bound on the number of nodes split during the construction of a p -node ($p > 2$) B-tree of order 3.

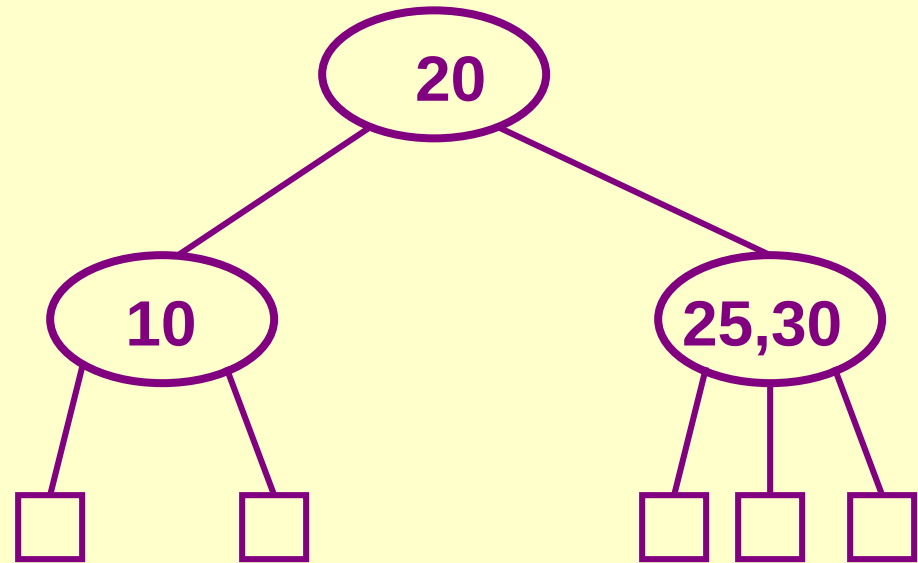
Insertion order: 10, 30, 20(split), 25, 28(split)



(a) $p=1, s=0$

After insert 10, 30

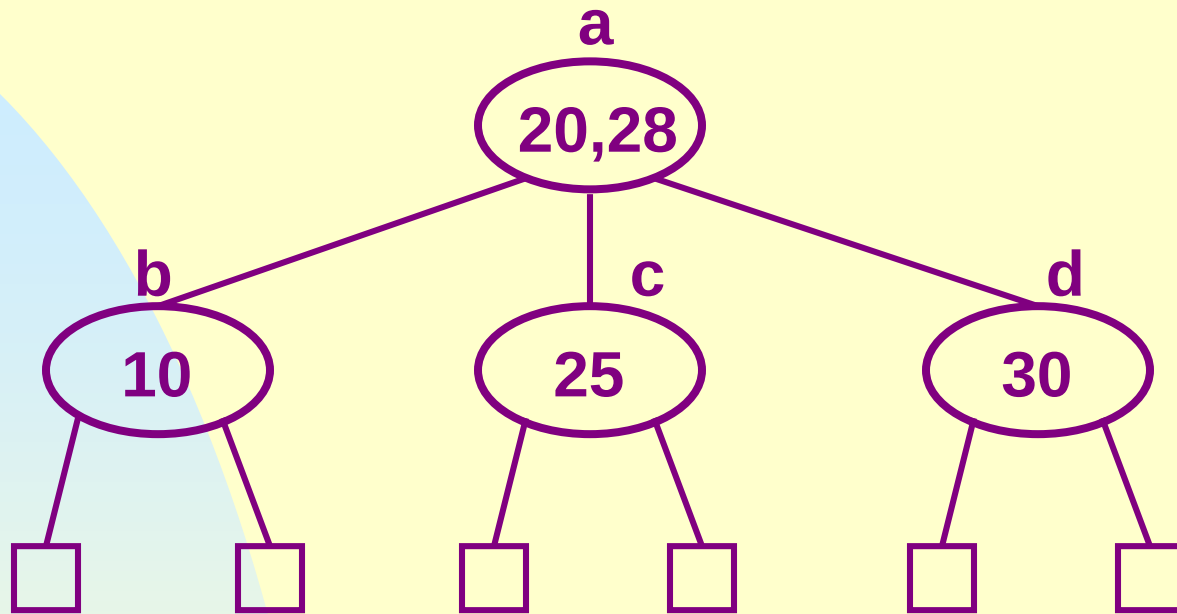
s is the number
of the nodes split



(b) $p=3, s=1$

After insert 20(split), 25

Note that there is no B-tree with $p=2$.



(c) $p=4$, $s=2$

After insert 28(split)

✓ A B-tree of order m with p nodes has at least
 $1 + (\lceil m/2 \rceil - 1)(p - 1)$ elements.
 (root) (other nodes)

✓ The average number of splits s_{avg} for an insertion is as follows:

$$s_{avg} = (\text{total number of splits } s) / N$$

$$\leq (p - 2) / \{1 + (\lceil m/2 \rceil - 1)(p - 1)\} < 1 / (\lceil m/2 \rceil - 1)$$

For $m=200$, $s_{avg} < 1/99$.

✓ The total number of disk accesses in N times of insertion is $Nh + (s - 1) + 2 + N$. For an insertion, h times disk accesses for read, and 1 time for write when no node split.

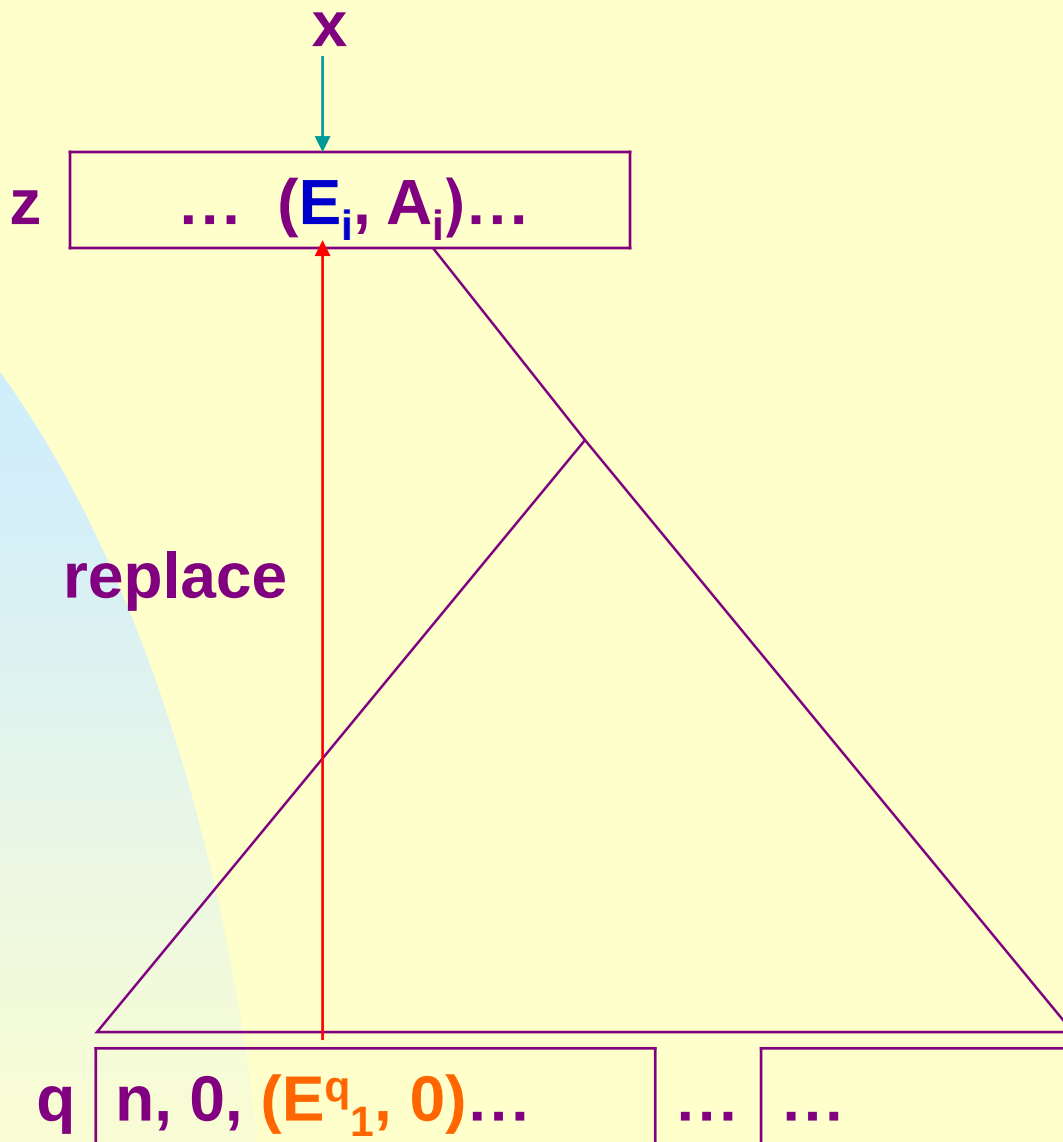
✓ So the average disk access for an insertion is

$$h + (s + 1) / N + 1 < h + 1 / (\lceil m/2 \rceil - 1) + 1 \approx h + 1.$$

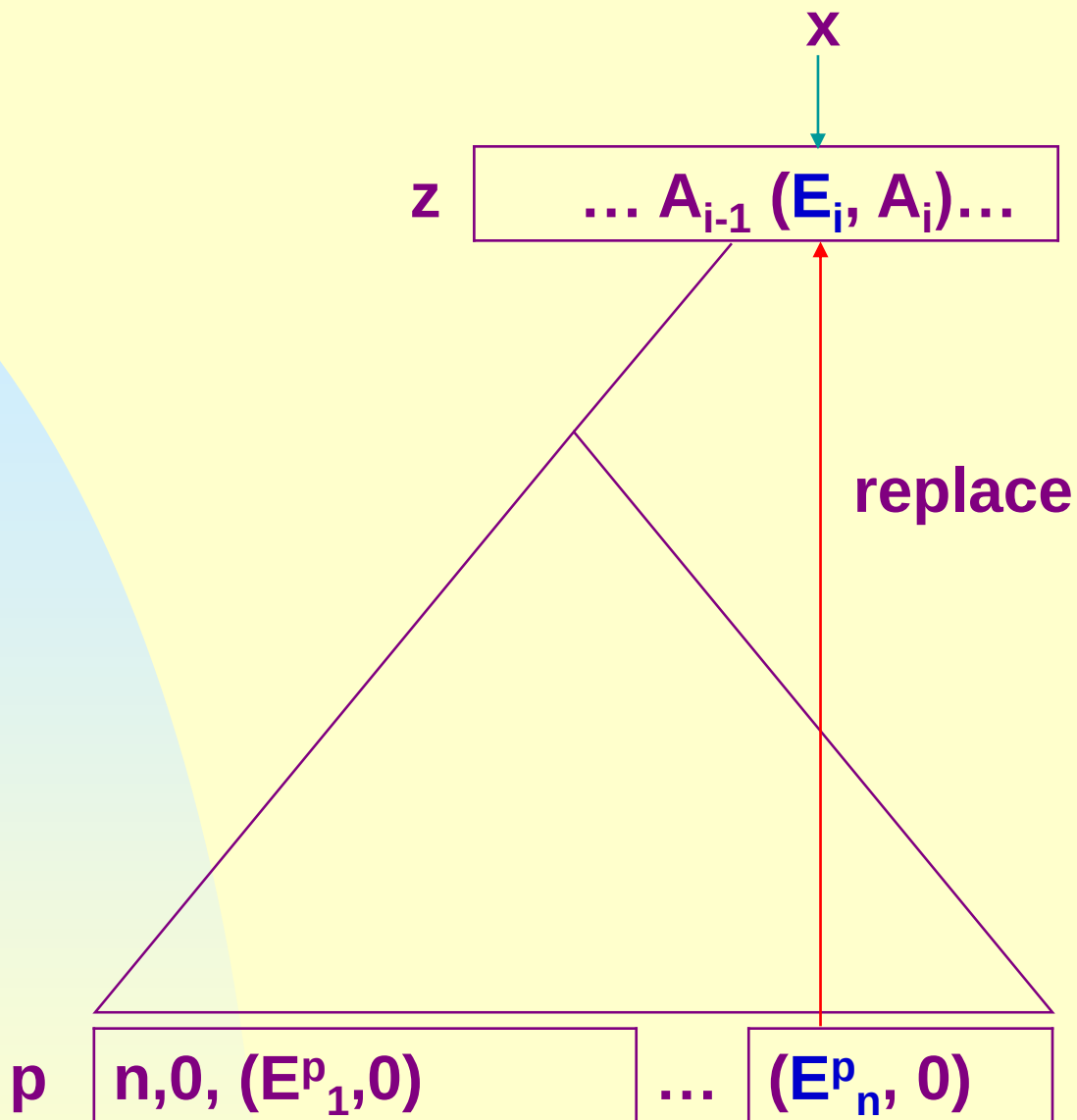
11.2.4 Deletion from a B-Tree

Suppose the key of the element to be deleted is x .

- ✓ First, search for x . If x is not found, no element is to be deleted.
- ✓ If x is found in a **nonleaf node** z and suppose that x is the i -th key in z (i.e. $x = E_i.K$), then E_i may be replaced by either **the element with the smallest key in the subtree A_i** or **the element with the largest key in the subtree A_{i-1}** . Both of these elements must be in **leaf nodes**.
- ✓ The deletion from a nonleaf node is thus transformed into a **deletion from a leaf**, as shown in the next two slides.



E_i is replaced by the element with
the smallest key in the subtree A_i .



E_i is replaced by the element with
the largest key in the subtree A_{i-1} .

■ 4 cases after deleting from a leaf node p:

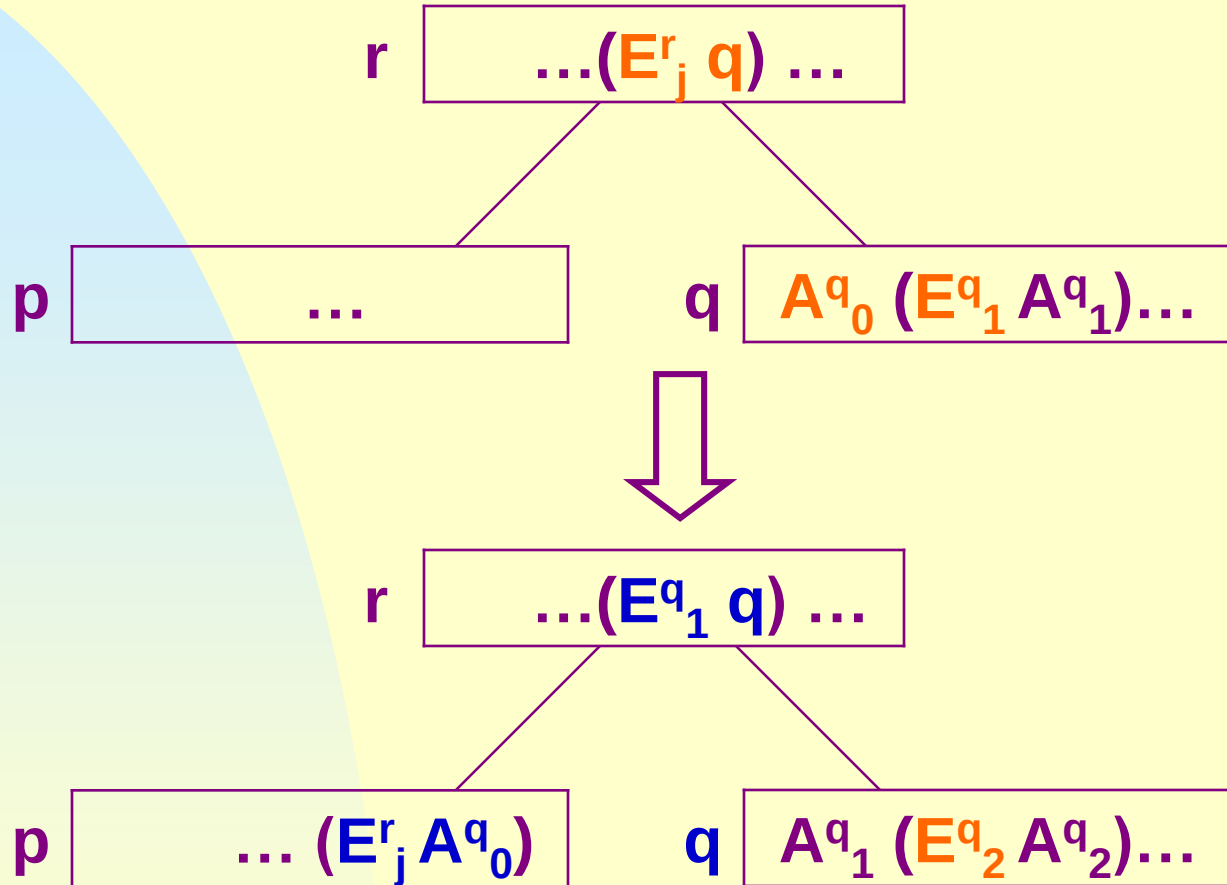
(1) p is also the **root**: If p is still **left with at least 1 key**, p is written to disk, and we are done; Otherwise the B-tree is **empty**.

Otherwise:

(2) p has **at least $\lceil m/2 \rceil - 1$ keys left**. The modified leaf node is written to disk, and we are done.

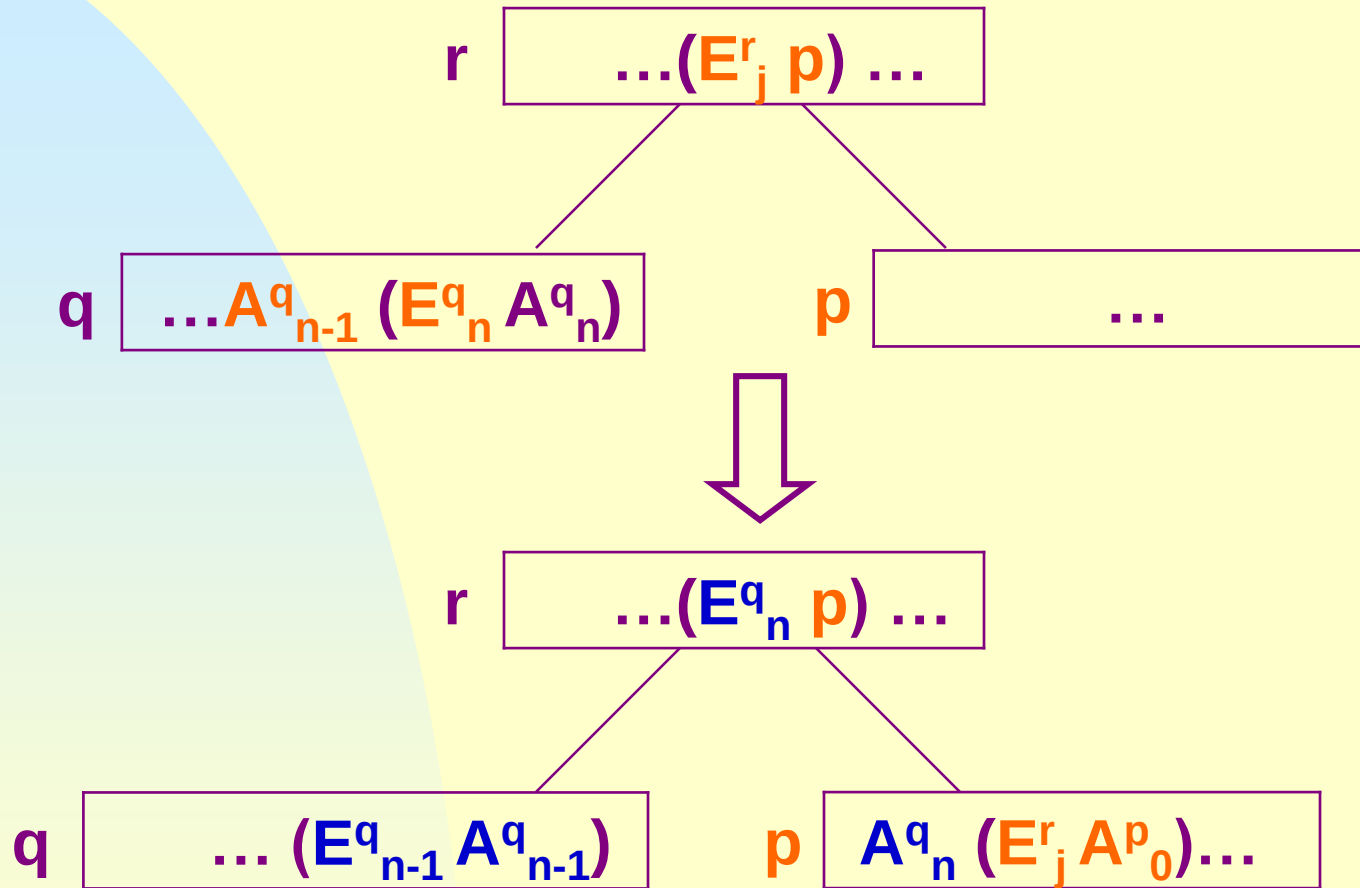
(3) (**rotation**) p only has $\lceil m/2 \rceil - 2$ keys left, and its **nearest sibling**, q, has at least $\lceil m/2 \rceil$ keys. The rotation is performed as shown in the next two slides (suppose q be the **nearest right or left sibling of p**, r be parent of p and q).

q is the nearest **right sibling** of p.



The changed p, q and r are written to disk, done.

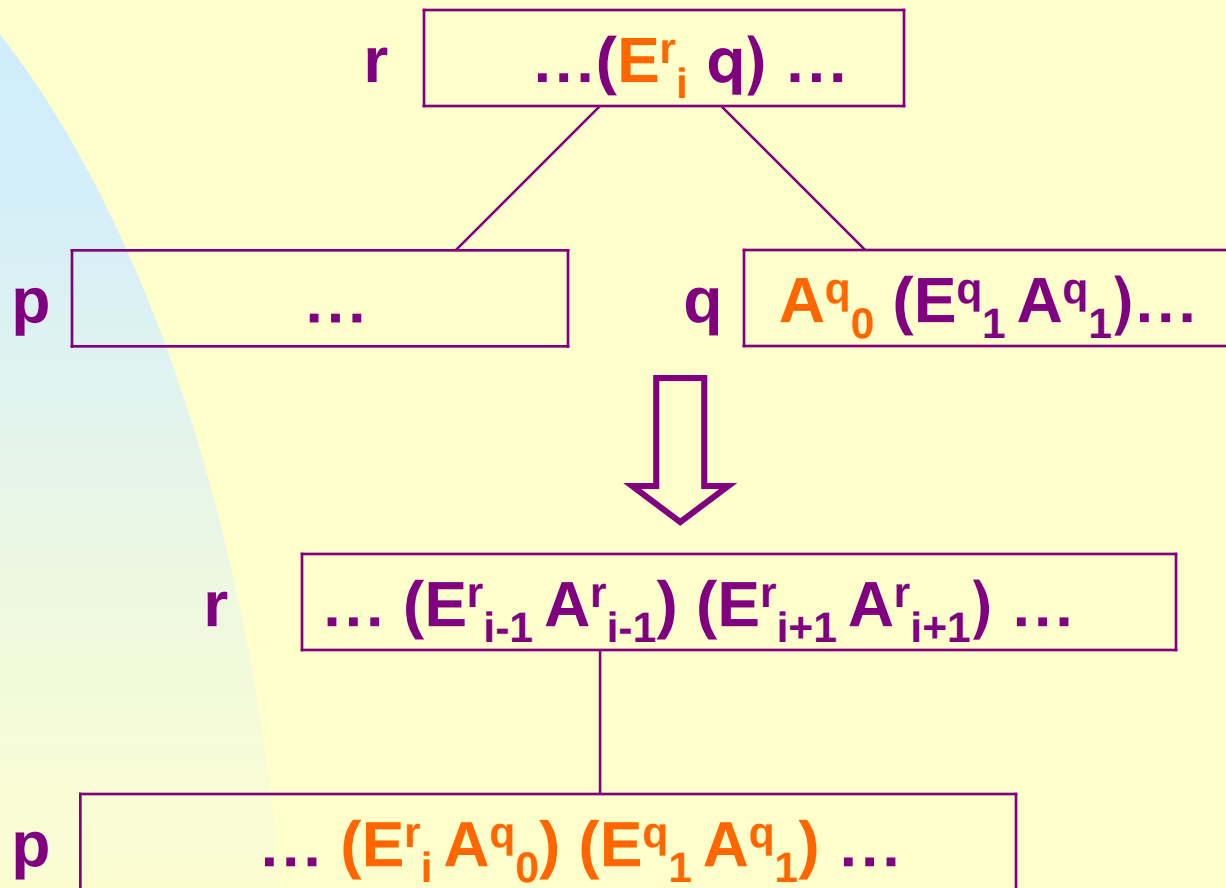
q is the nearest **left sibling** of p .



The changed p , q and r are written to disk, done.

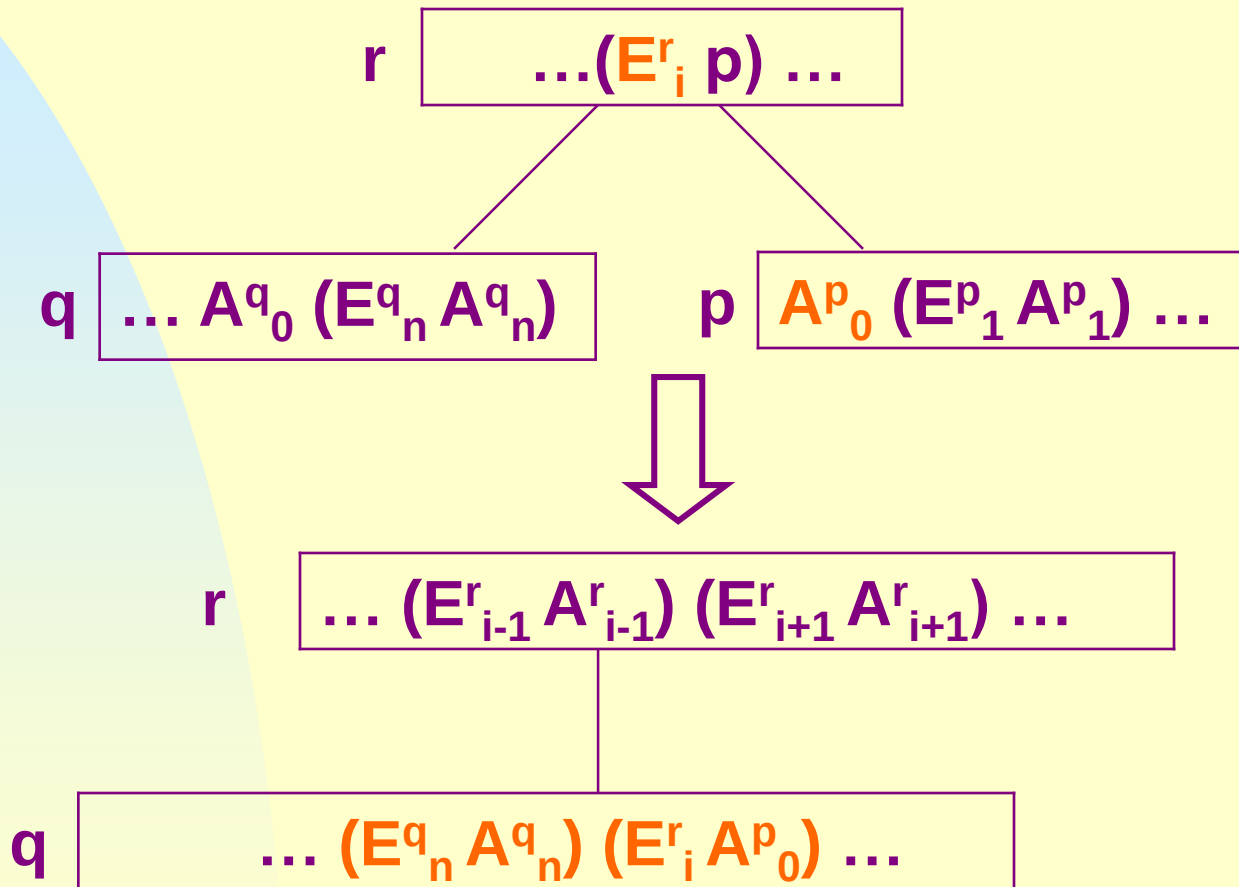
(4) (**combination**) p has $\lceil m/2 \rceil - 2$ keys left, and its nearest sibling q has $\lceil m/2 \rceil - 1$ keys. Nodes p and q and **in-between element** E_i in the parent r are combined to form a single node. The **combination** is shown in the next two slides.

q is the **right sibling** of p .



Note that $A_{i-1}^r = p$.

q is the **left sibling** of p .



Note that $A_{i-1}^r = q$.

- ✓ Now the combined node has $(\lceil m/2 \rceil - 2) + (\lceil m/2 \rceil - 1) + 1 = 2\lceil m/2 \rceil - 2 \leq m - 1$ keys. Then it is written to disk.
- ✓ The number of keys in the parent, r , has been reduced by 1.
- ✓ If r does not become deficient (i.e., it has at least 1 element left if it is the root and $\lceil m/2 \rceil - 1$ elements left if it is not the root), the changed r is written to disk, and we are done.

✓ When r becomes deficient:

- If it is the root (no element left), it is discarded;
- Otherwise r is not the root and has $\lceil m/2 \rceil - 2$ keys:
 - We first attempt a **rotation** with one of its nearest siblings.
 - If this is not possible, a **combine** is done.
This process of combining can continue up until the children of the root are combined.

A brief description of the deletion algorithm

Program 11.3: Deletion from a B-tree that resides on disk

// Delete element with key x from a B-tree of order m

Search the B-tree for node p containing the element with key x ;

if (there is no such node p) **return**; // no element to delete

Let p be of the form: $n, A_0, (E_1, A_1), \dots, (E_n, A_n)$ and $E_i.K=x$;

if (p is not a leaf) {

 Replace E_i with the smallest key element in subtree A_i ;

write the altered p to disk;

 Let p be the leaf of A_i from which the smallest was taken;

 Let p be of the form: $n, A_0, (E_1, A_1), \dots, (E_n, A_n)$;

$i = 1$;

}

// the following deletes E_i from leaf node p

Delete (E_i, A_i) from p ; $n--$;

while $(n < \lceil m/2 \rceil - 1) \ \&\& \ (p \neq \text{root})$

if $(p \text{ has a nearest right sibling } q) \{$

Let q : $n_q, A^q_0, (E^q_1 A^q_1), \dots, (E^q_{n_q} A^q_{n_q})$;

Let r : $n_r, A^r_0, (E^r_1 A^r_1), \dots, (E^r_{n_r} A^r_{n_r})$ be parent of p and q ;

Let $A^r_j = q$ and $A^r_{j-1} = p$;

if $(n_q \geq \lceil m/2 \rceil) \{ \text{// rotation}$

$(E_{n+1}, A_{n+1}) = (E^r_j, A^q_0)$; $n = n + 1$; **// update** p

$E^r_j = E^q_1$; **// update** r

$(n_q, A^q_0, (E^q_1 A^q_1), \dots) = (n_q - 1, A^q_1, (E^q_2 A^q_2), \dots)$; **//update** q

write p, q, r to disk; **return**;

} **//end of inner if**

// combine nodes p, E^r_j , and q

$s = 2 * \lceil m/2 \rceil - 2$;

```
write s,  $A_0$ ,  $(E_1 A_1), \dots, (E_n A_n)$ ,  $(E_j^r A_0^q)$ ,  $(E_1^q A_1^q)$ ,  
...,  $(E_{nq}^q A_{nq}^q)$  to disk as node p;
```

```
// update for next iteration
```

```
free(q);
```

```
 $(n, A_0, \dots) = (n_r - 1, A_0^r, \dots, (E_{j-1}^r A_{j-1}^r), (E_{j+1}^r A_{j+1}^r) \dots);$ 
```

```
p=r;
```

```
} // end of if p has a nearest right sibling
```

```
else { // p must have a left sibling, this is symmetric to the
```

```
    // case where p has a right sibling
```

```
} // end of if-else and while
```

```
if (n) write p:  $(n, A_0, (E_1 A_1), \dots, (E_n A_n));$ 
```

```
else { root =  $A_0$ ; free(p); } // change root, always combine to left
```

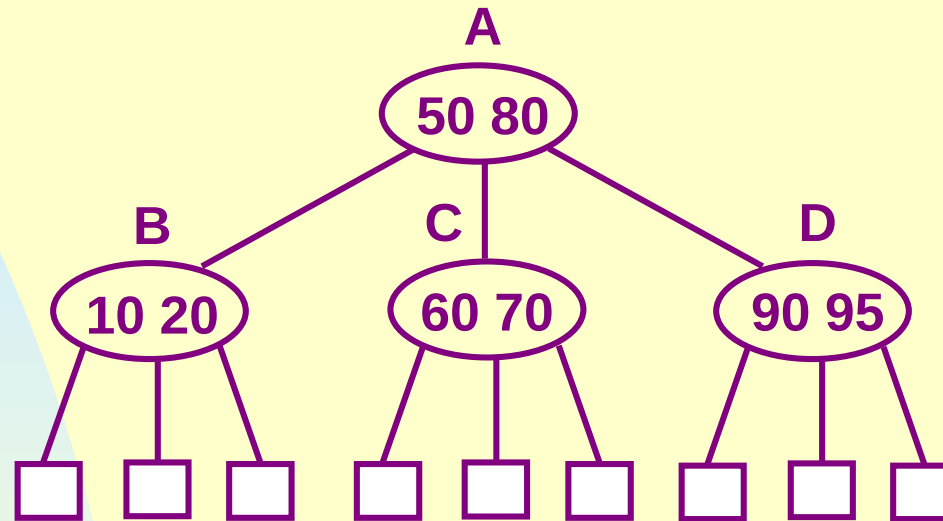
Analysis of B-tree Deletion:

- ✓ Once again, we assume that disk nodes accessed during the **downward search pass** may be **stored in main memory**, so they do not need to be re-accessed from the disk during the upward restructuring.
- ✓ For a B-tree of height h (external nodes is in level $h+1$), **h disk accesses** are made to find the corresponding **leaf node**.
- ✓ In the worst, a **combine** takes place at each of **the last $h-2$ nodes** on the root-to-leaf path, and a **rotation** takes place at the **2nd node** on this path. (**$h-2$ combines + 1 rotation**)

- Note if a **combine** takes place at the 2nd node on this path, there are only 3 disk accesses (one for find the nearest sibling and two for writing out)
- The **combines** need **$h-2$** disk accesses to retrieve the nearest sibling for each node and another **$h-2$** accesses for writing out. (**$(h-2)+(h-2)$** times of disk accesses)
- The **rotation** needs **1** disk access to read a nearest sibling and **3** to write out. (**$1+3$** times of disk accesses)
- So, in the worst case, the total number of disk accesses is **$3h$** .

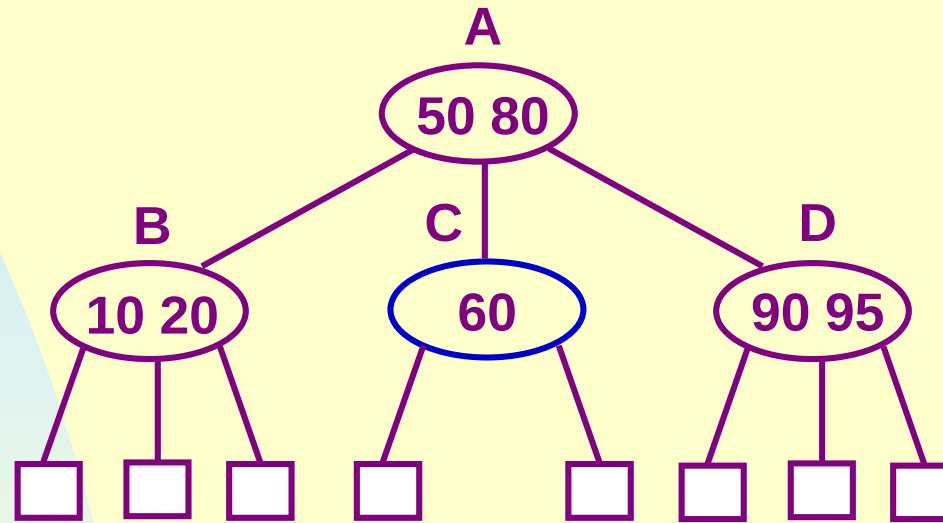
$$3h = h + (h-2) + (h-2) + 1 + 3.$$

Example 11.2:



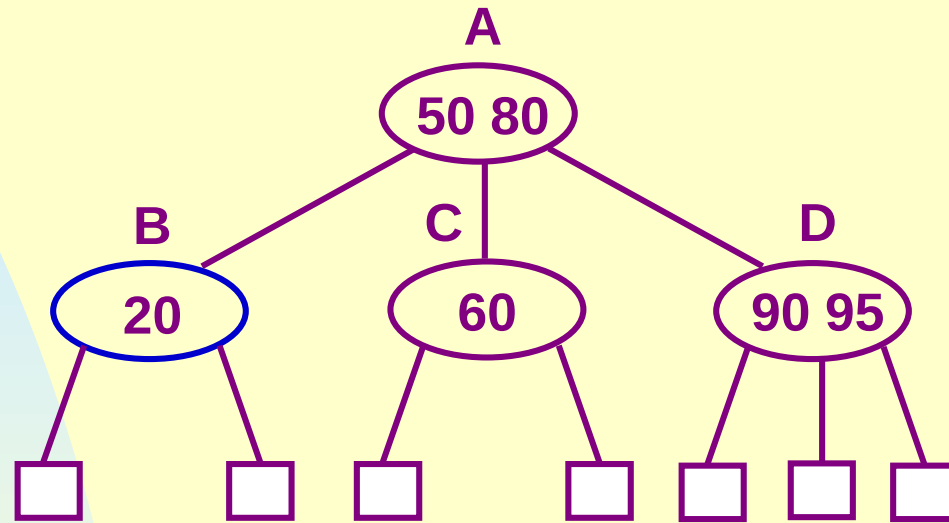
An initial 2-3 tree

Deletion order: 70, 10, 60, 95, 90, 20.



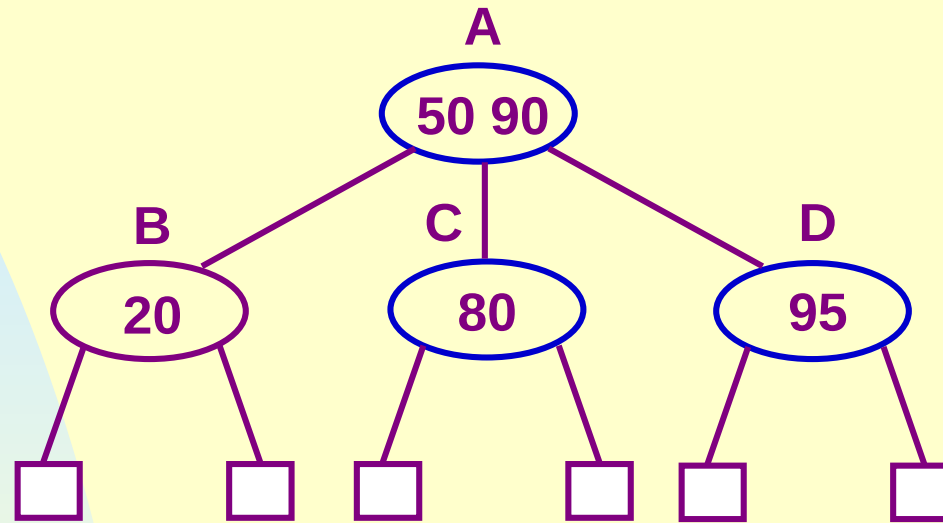
70 deleted

Deletion order: 70, 10, 60, 95, 90, 20.



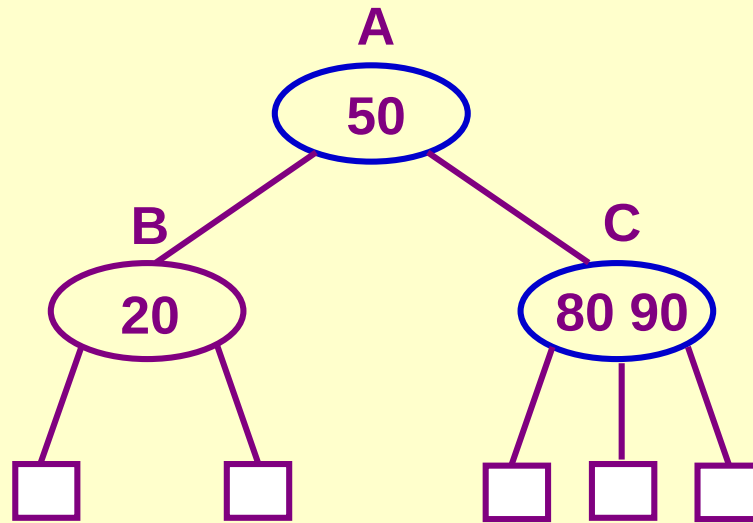
10 deleted

Deletion order: 70, 10, 60, 95, 90, 20.



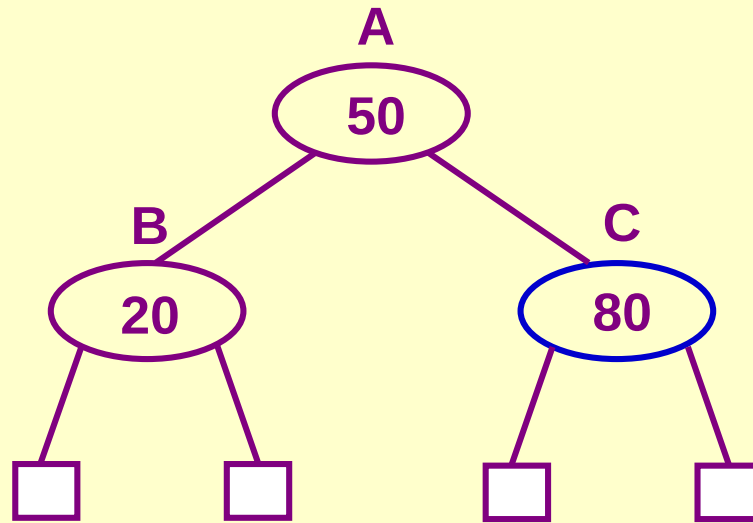
60 deleted (rotation)

Deletion order: 70, 10, 60, 95, 90, 20.



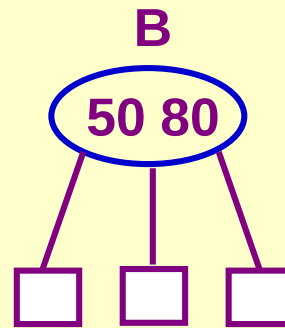
95 deleted (combine)

Deletion order: 70, 10, 60, 95, 90, 20.



90 deleted

Deletion order: 70, 10, 60, 95, 90, 20.



20 deleted

Deletion order: 70, 10, 60, 95, 90, 20.

11.3 B⁺-Trees

Definition--- A B⁺-tree of order m is a tree that either is empty or it has two types of nodes—**index and data** and satisfies the following properties:

- (1) All **data nodes** are at the same level and are leaves. Data nodes **contains elements**.
- (2) The **index nodes** define a B-tree of order m ; each index node **has keys but not elements**.
- (3) Each **index node** has the structure: $n, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_n, A_n)$ where A_i ($0 \leq i \leq n < m$), are pointers to subtrees, and K_i ($1 \leq i \leq n < m$) are keys. If we let $K_0 = -\infty$ and $K_{n+1} = \infty$, then **All elements in subtree A_i have the key greater than or equal to K_i and less than K_{i+1} for $0 \leq i \leq n$.**

Differences between B⁺-tree and B-tree

A B⁺-tree is a close cousin of the B-tree. The essential differences between them are:

- The **index nodes** and **data nodes** of a B⁺-tree correspond to the **internal nodes** and **external nodes** of a B-tree, respectively.
- The **index nodes** store keys (not elements) and pointers, and the **data nodes** store elements (together with their keys but no pointers), while **an internal node** store both elements and pointers.
- The **data nodes** are linked together to form a doubly linked list, while **the external nodes** are not physically realized.

B+-tree example

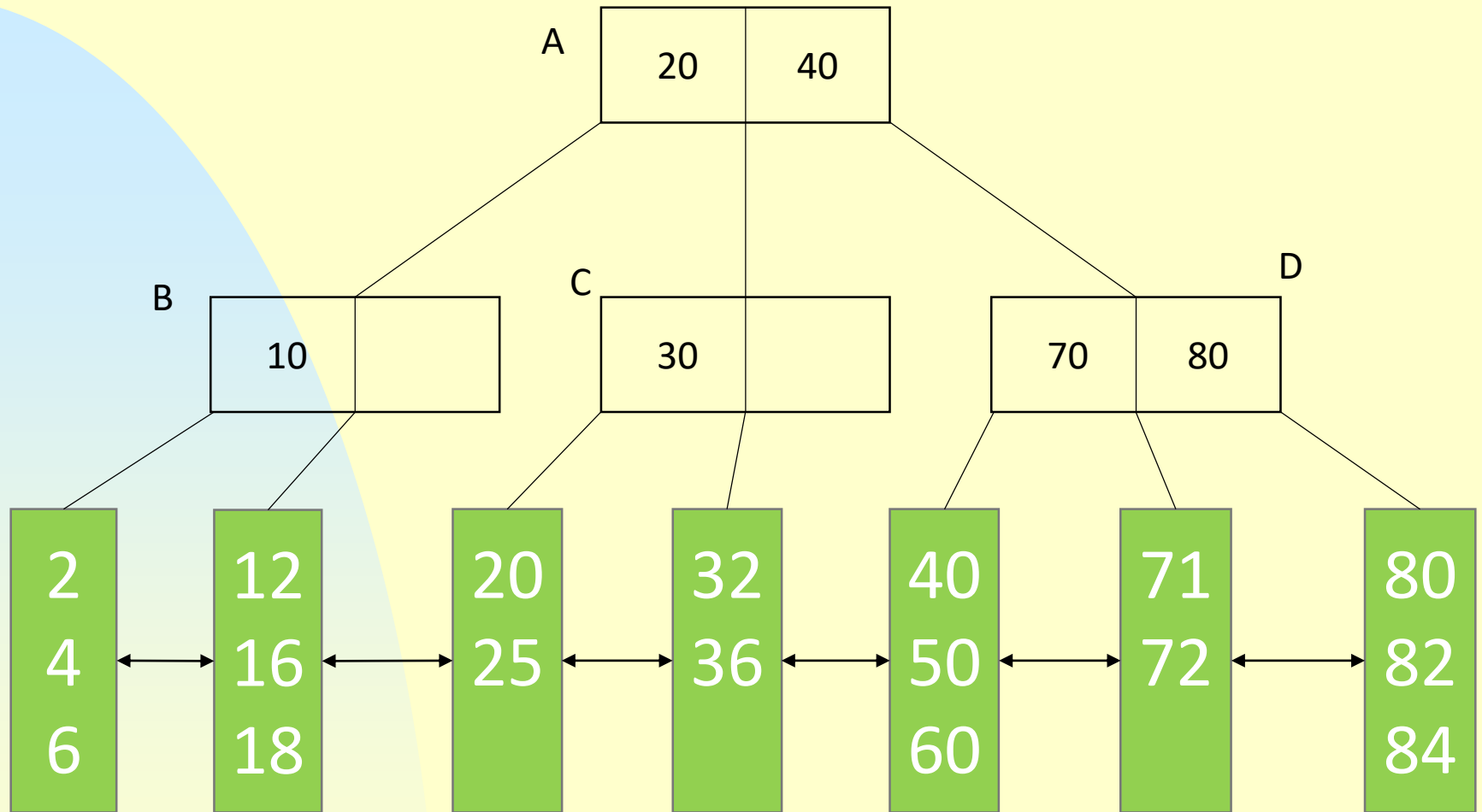


Fig 11.11 A B⁺-tree of order 3

Searching a B⁺-tree

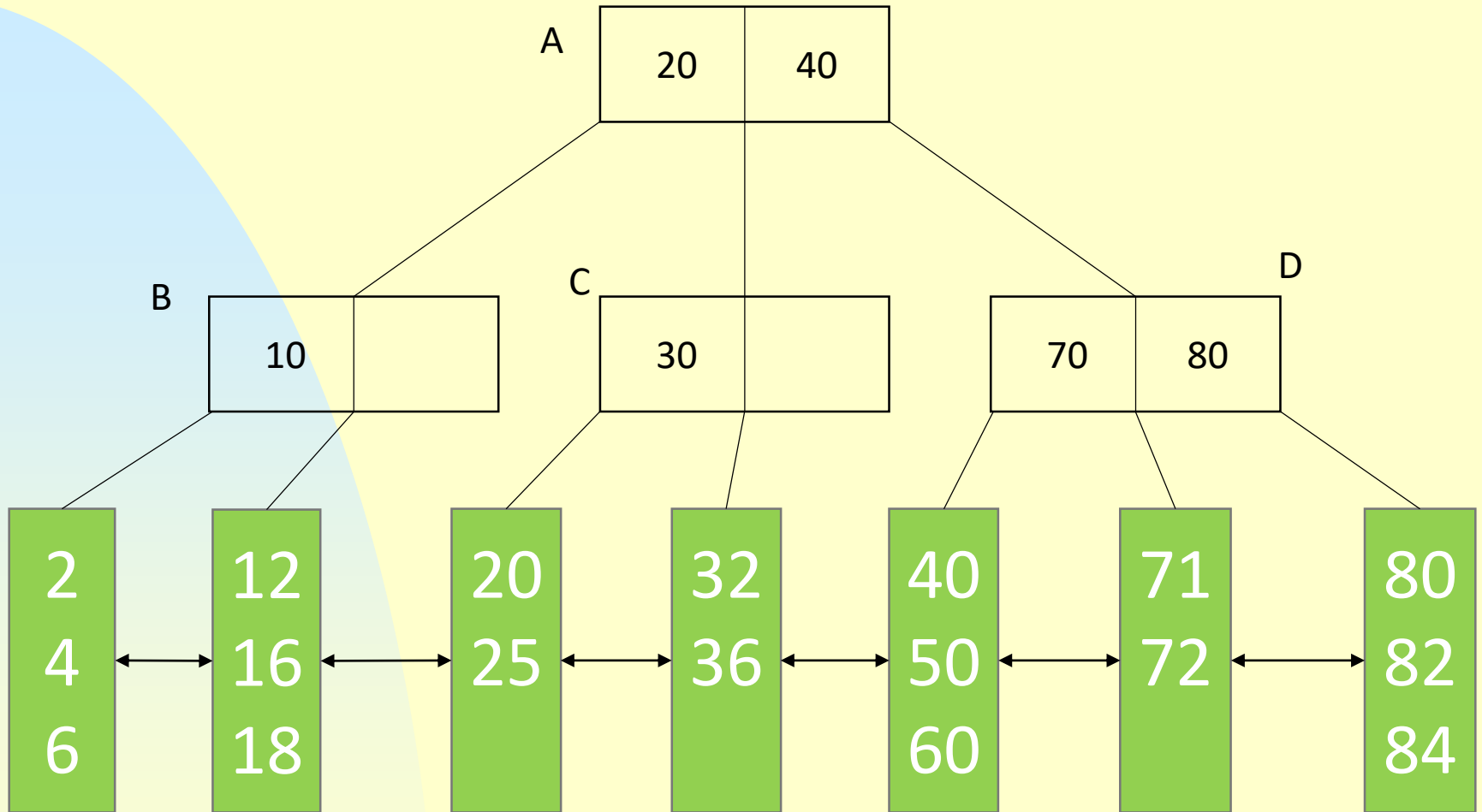
- B⁺-trees support two types of searches---**exact match and range.**
- ✓ **Exact match search:** For example, search for the element with the key 32. **Begin at A**; $20 \leq 32 < 40$ moves to C; $30 \leq 32$, search the data node that contains the elements with keys 32 and 36.
- ✓ **Range search:** For example, search for all the elements with keys in range [16, 70]. **First**, an exact match search for 16 (the 2nd data node); **Next**, travel down the doubly linked list of data nodes until we reach a data node that has an element with key **exceeds 70 or the end of the list.**

Program 11.4 Exact searching a B⁺-tree

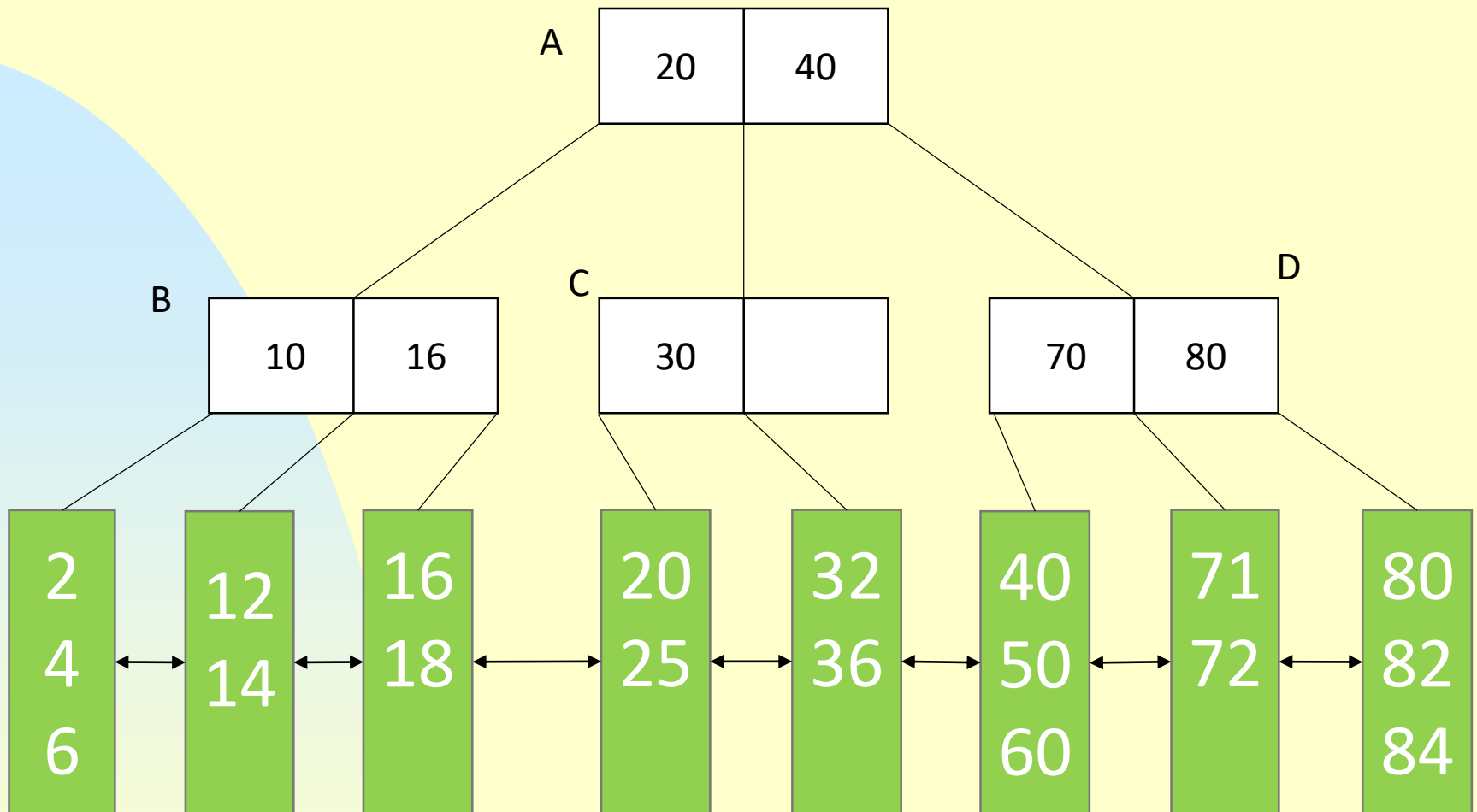
```
//Search a B+-tree of an element with key x.  
//Return the element if found. Return NULL otherwise.  
if the tree is empty return NULL;  
K0=-MAXKEY;  
for (*p=root; p is an index node; p=Ai)  
{  
    Let p have the format n, A0, (K1, A1), (K2, A2), ..., (Kn, An);  
    Kn+1=MAXKEY;  
    Determine i such that  $K_i \leq x < K_{i+1}$ ;  
}  
//search the data node p  
Search p for an element E with key x;  
if such an element is found return E;  
else return NULL;
```

Insertion into a B⁺-tree

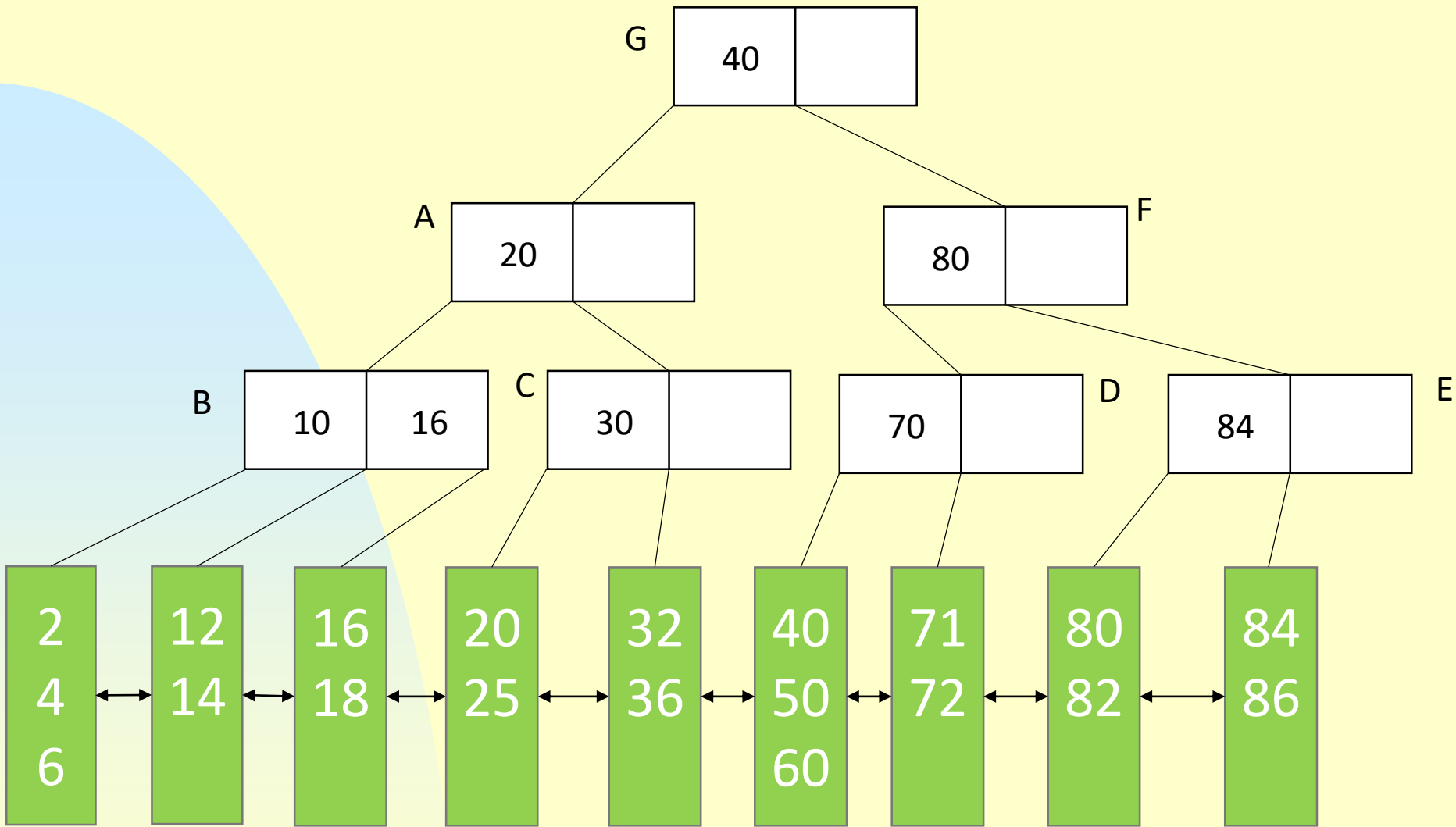
The capacity of a data node is 3.



Initial (Fig 11.11)
Insert order: 14, 86



14 inserted
Insert order: 14, 86

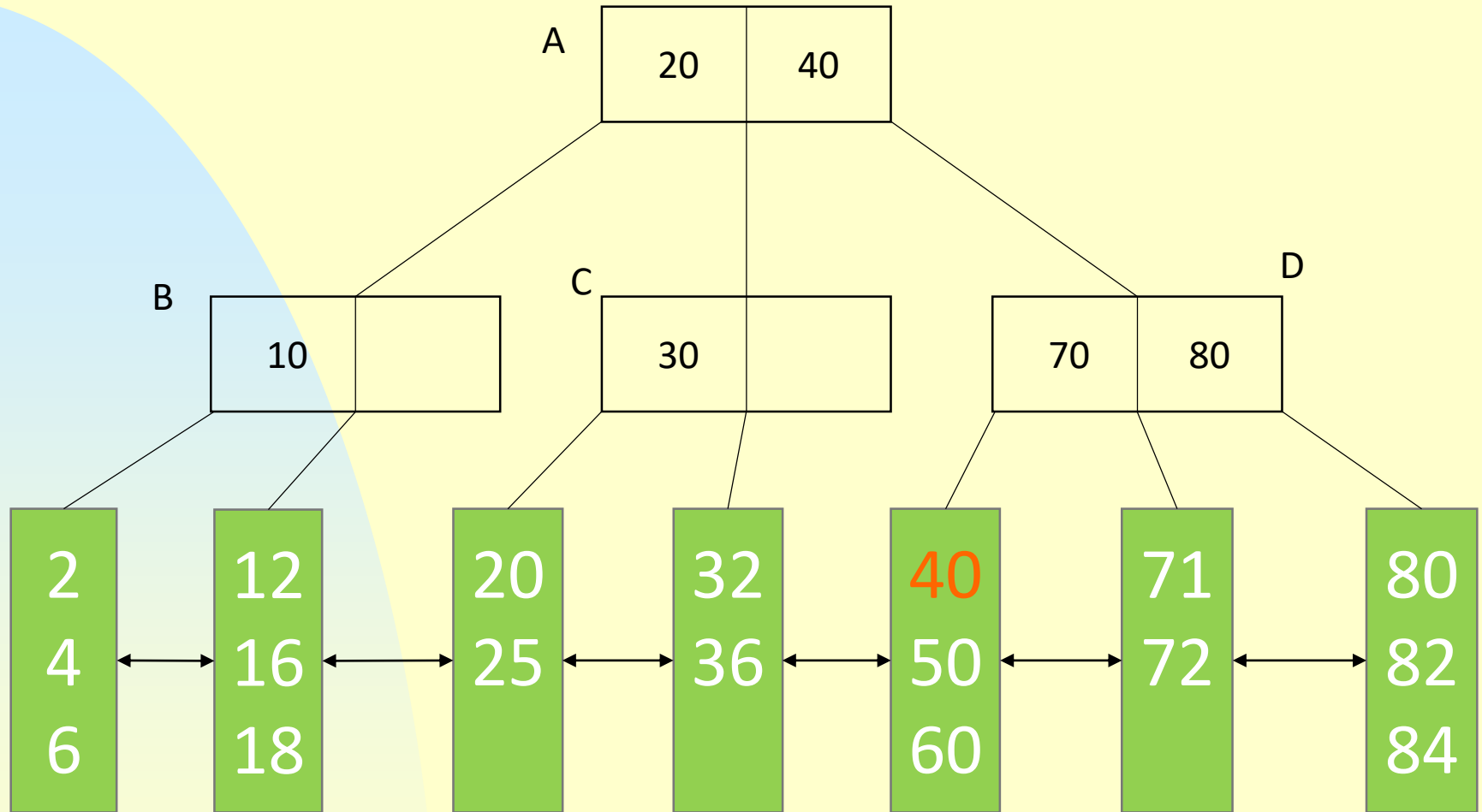


86 inserted
Insert order: 14, 86

Deletion from a B⁺-tree

- We only need to consider **deletion from a leaf** since elements are stored only in the leaves of a B⁺-tree.
- More **constraints about data nodes**:
 - ✓ a non-root data node is deficient iff it has **fewer than $\lceil c/2 \rceil$ elements**;
 - ✓ a root data node (i.e., No index node in the B⁺ tree) is deficient iff it is **empty**.
- In the above and following examples, **c=3**.

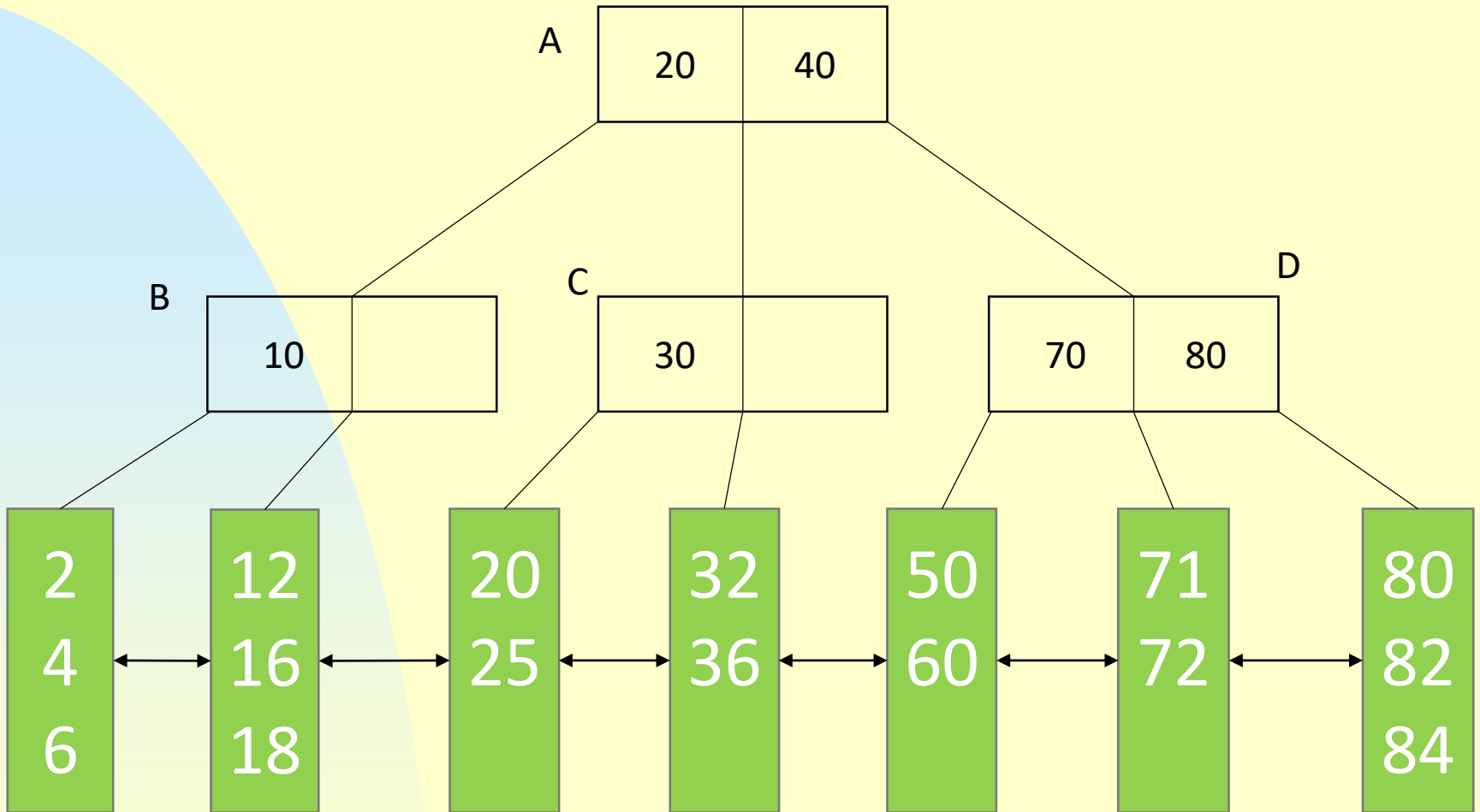
Deletion from a B⁺-tree (case a)



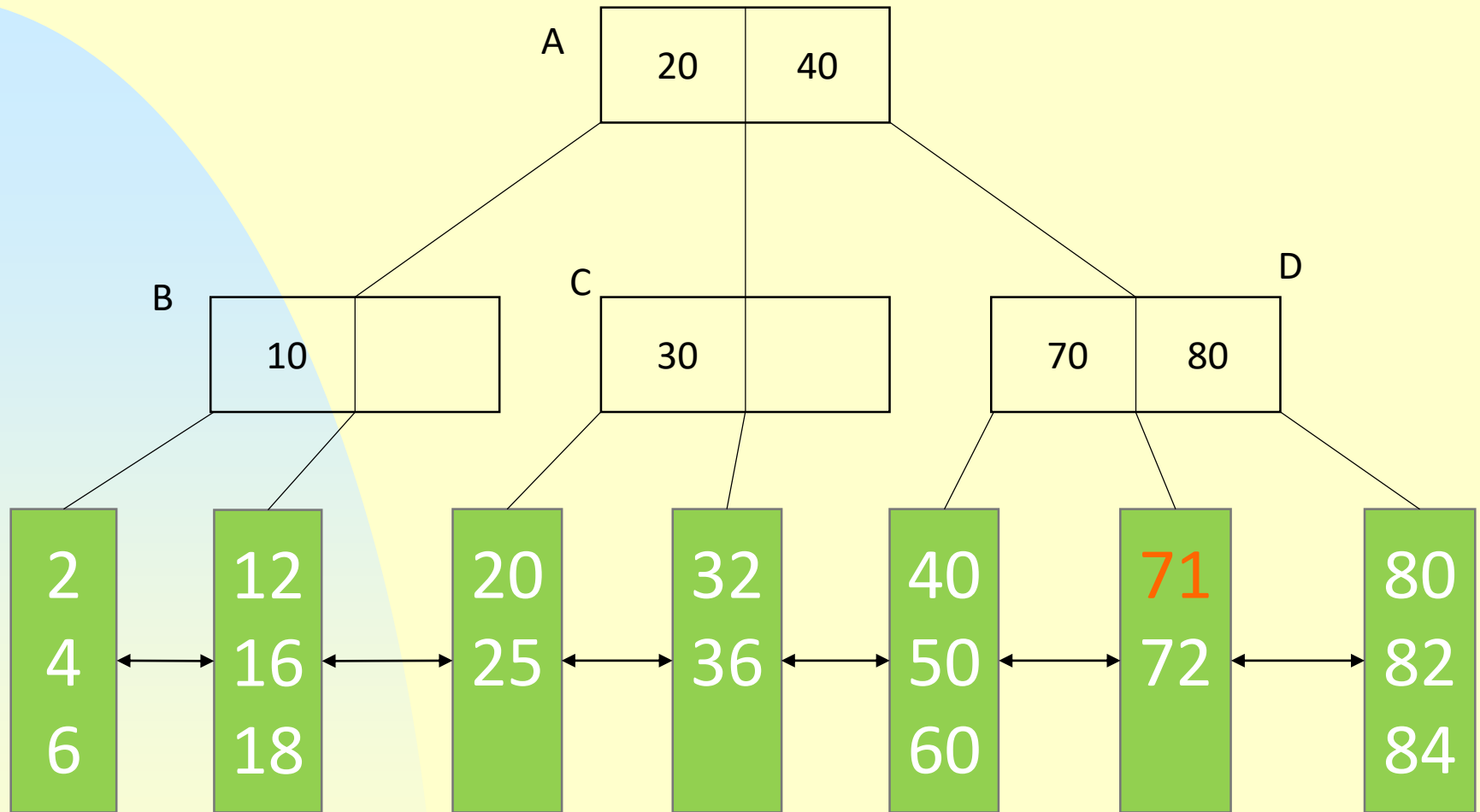
Deletion: 40

(a) When the deletion of an element **doesn't result in a deficient data node**, no index node should be changed.⁶³

Deletion from a B⁺-tree (case a)



Deletion from a B⁺-tree (case b: “borrow”)

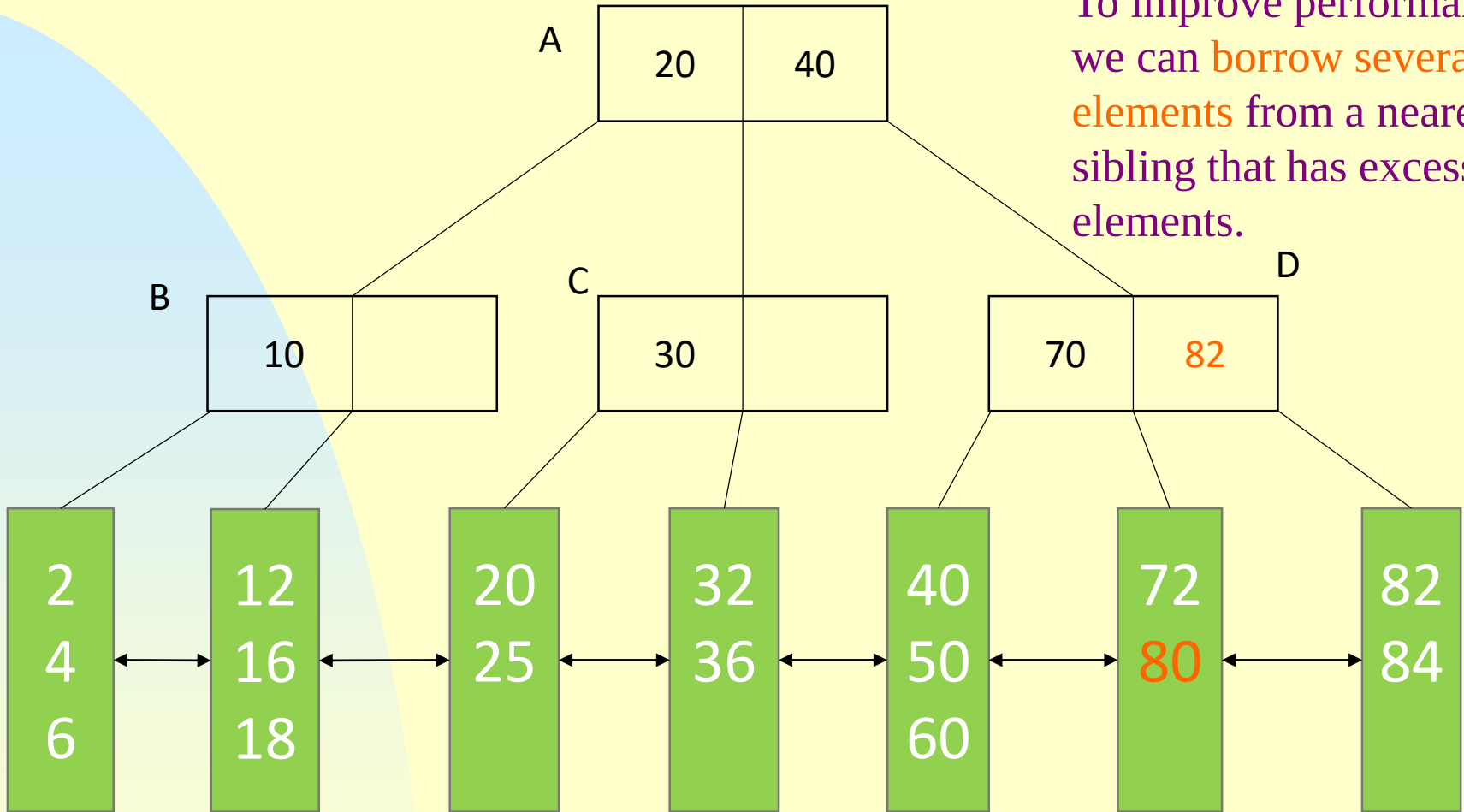


Deletion: 71

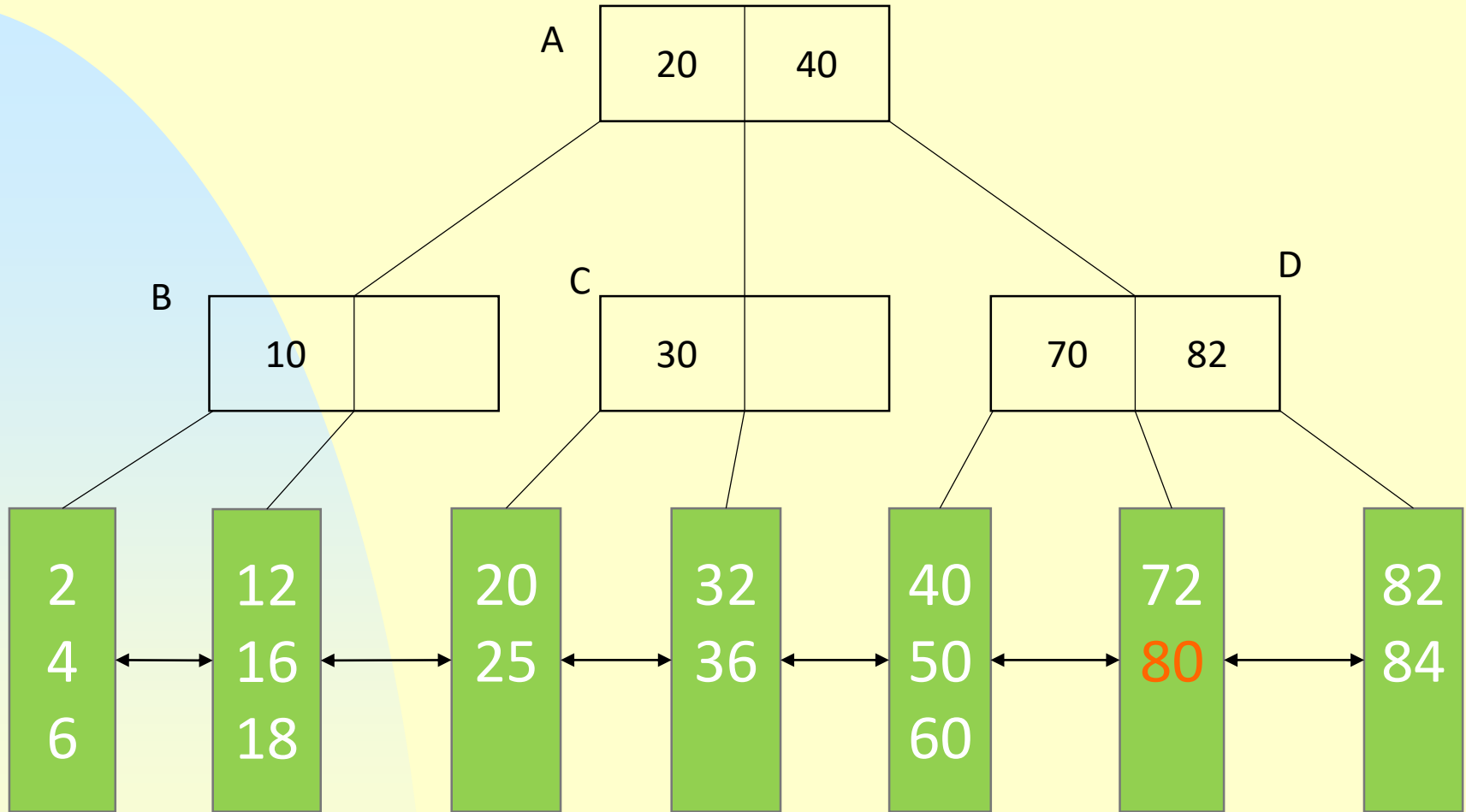
(b) We should update the in-between key in the parent to the smallest remaining key in the right sibling.

Deletion from a B⁺-tree (case b: “borrow”)

To improve performance, we can **borrow** several **elements** from a nearest sibling that has excess elements.



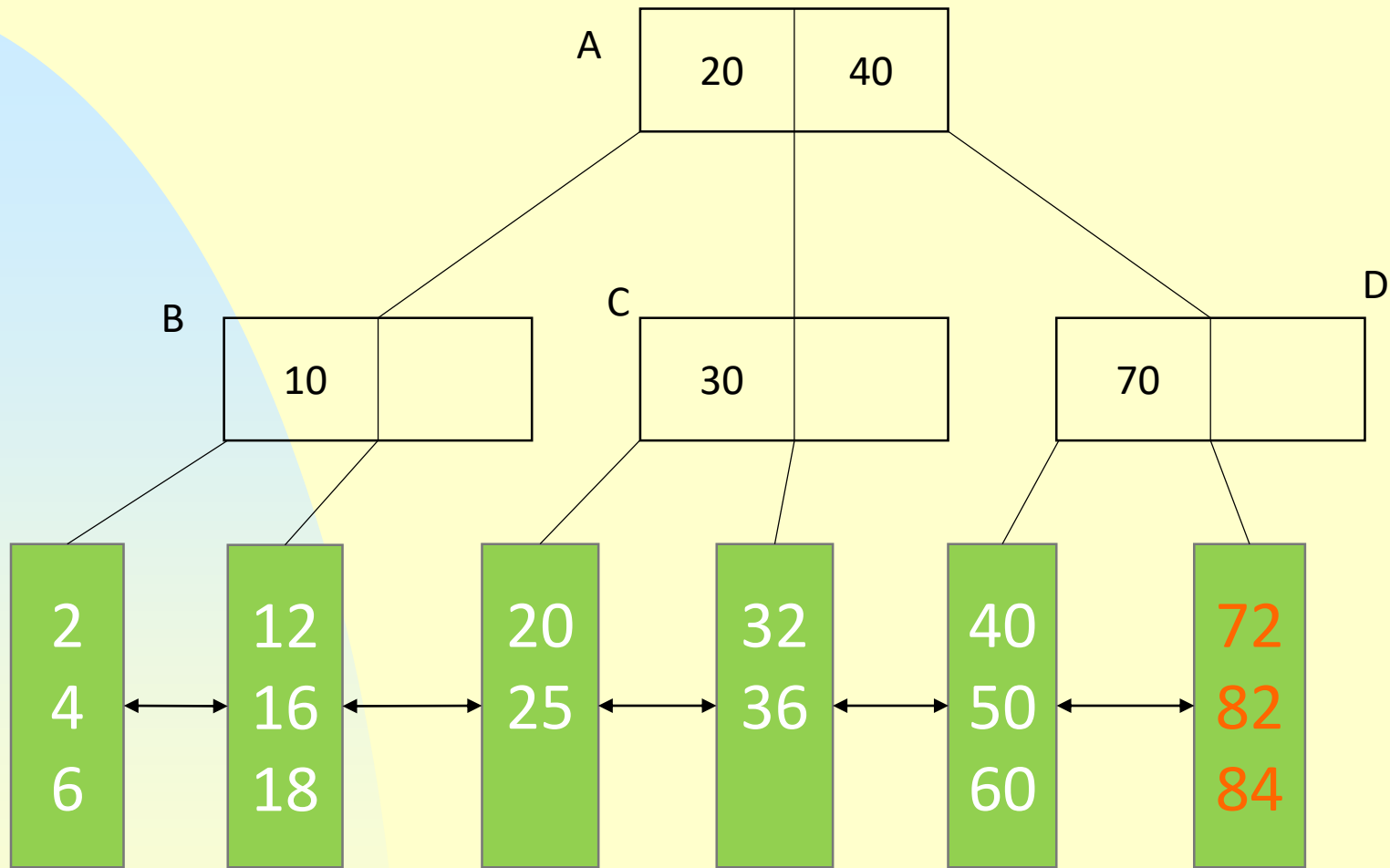
Deletion from a B⁺-tree (case c: “combine”)



Deletion: 80

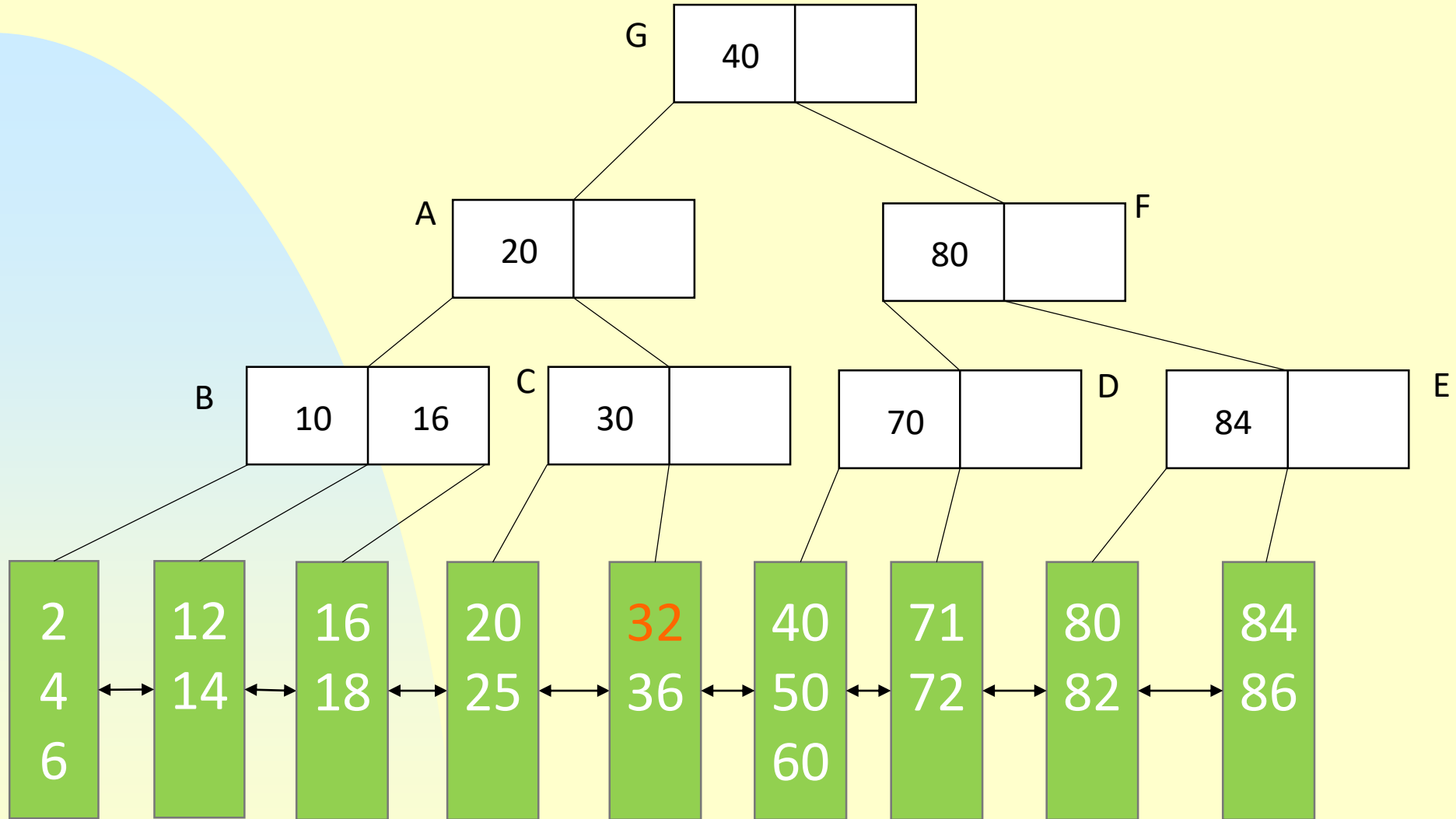
(c) The in-between key in the parent should be deleted.

Deletion from a B⁺-tree (case c: “combine”)

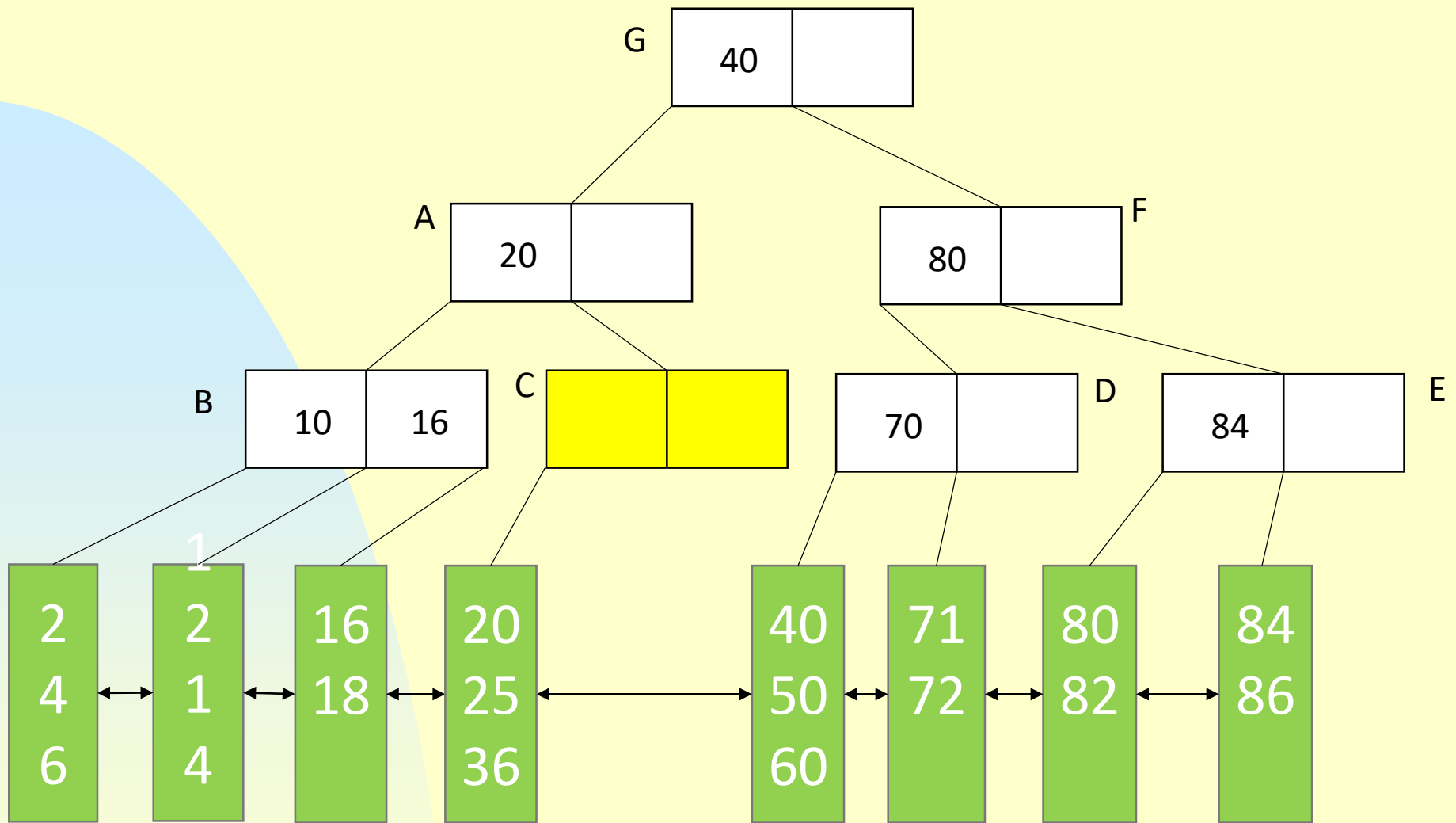


80 deleted

A complete example: Deletion from a B⁺-tree

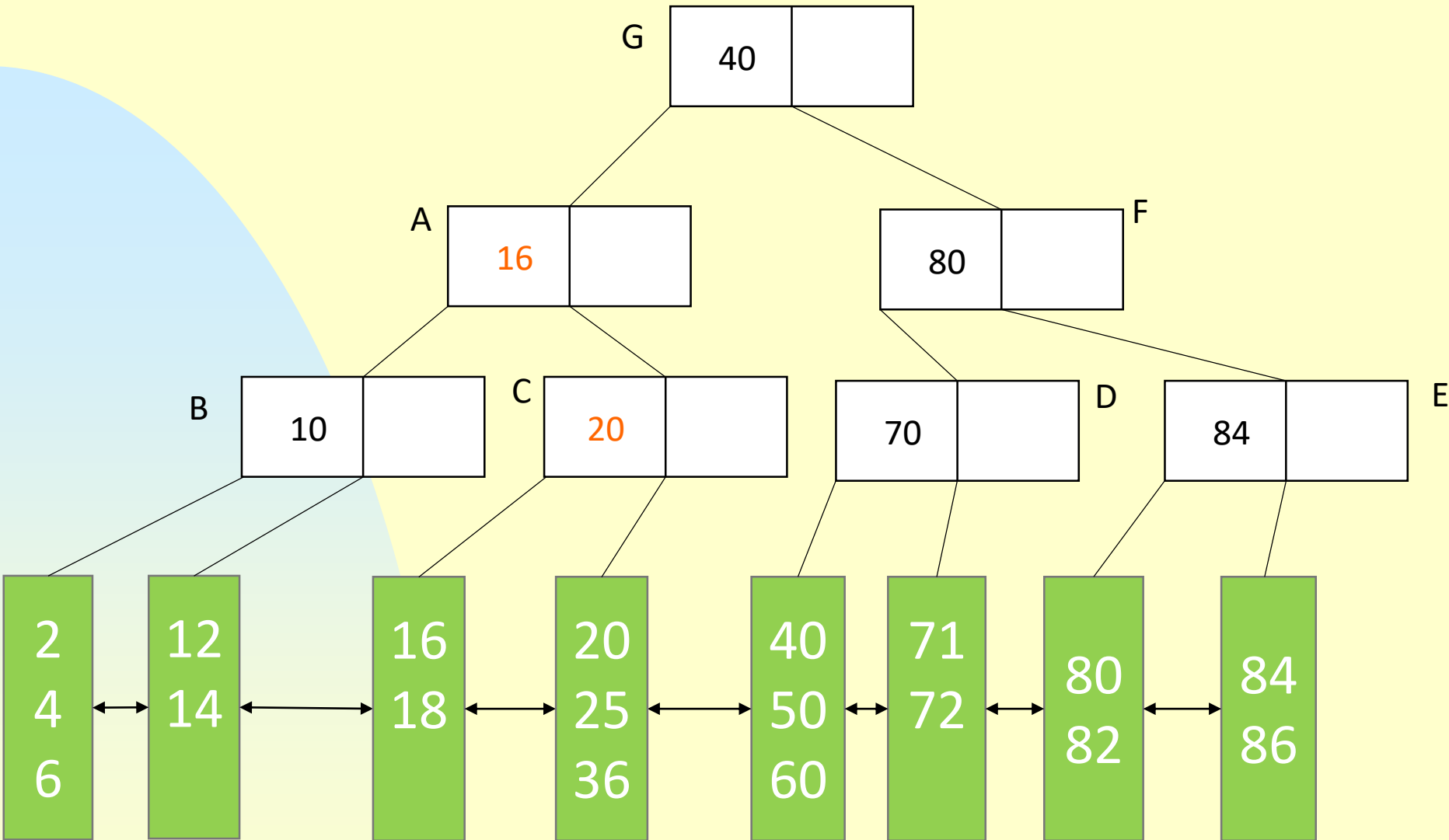


Deletion: 32, 86

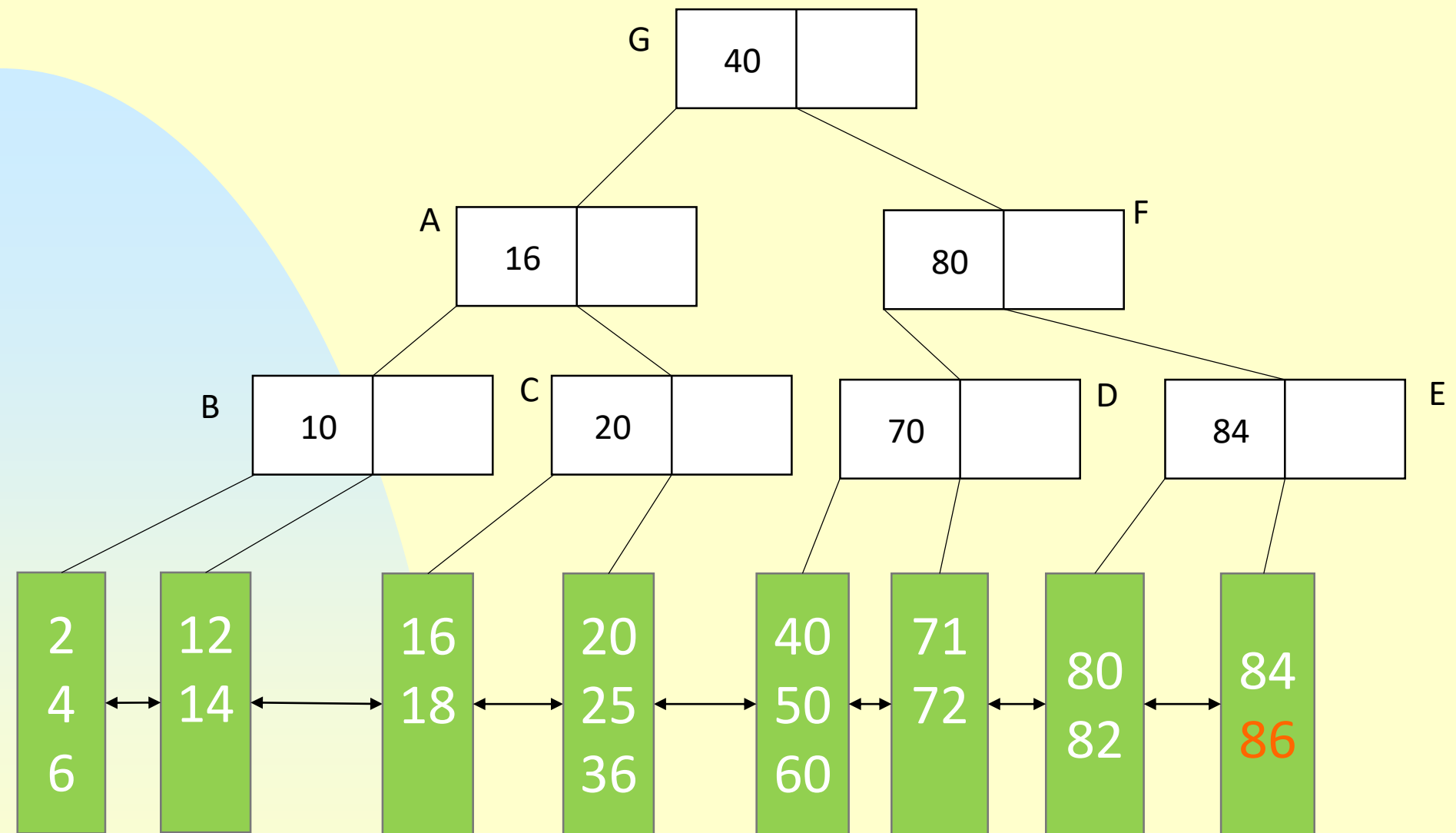


32 deleted

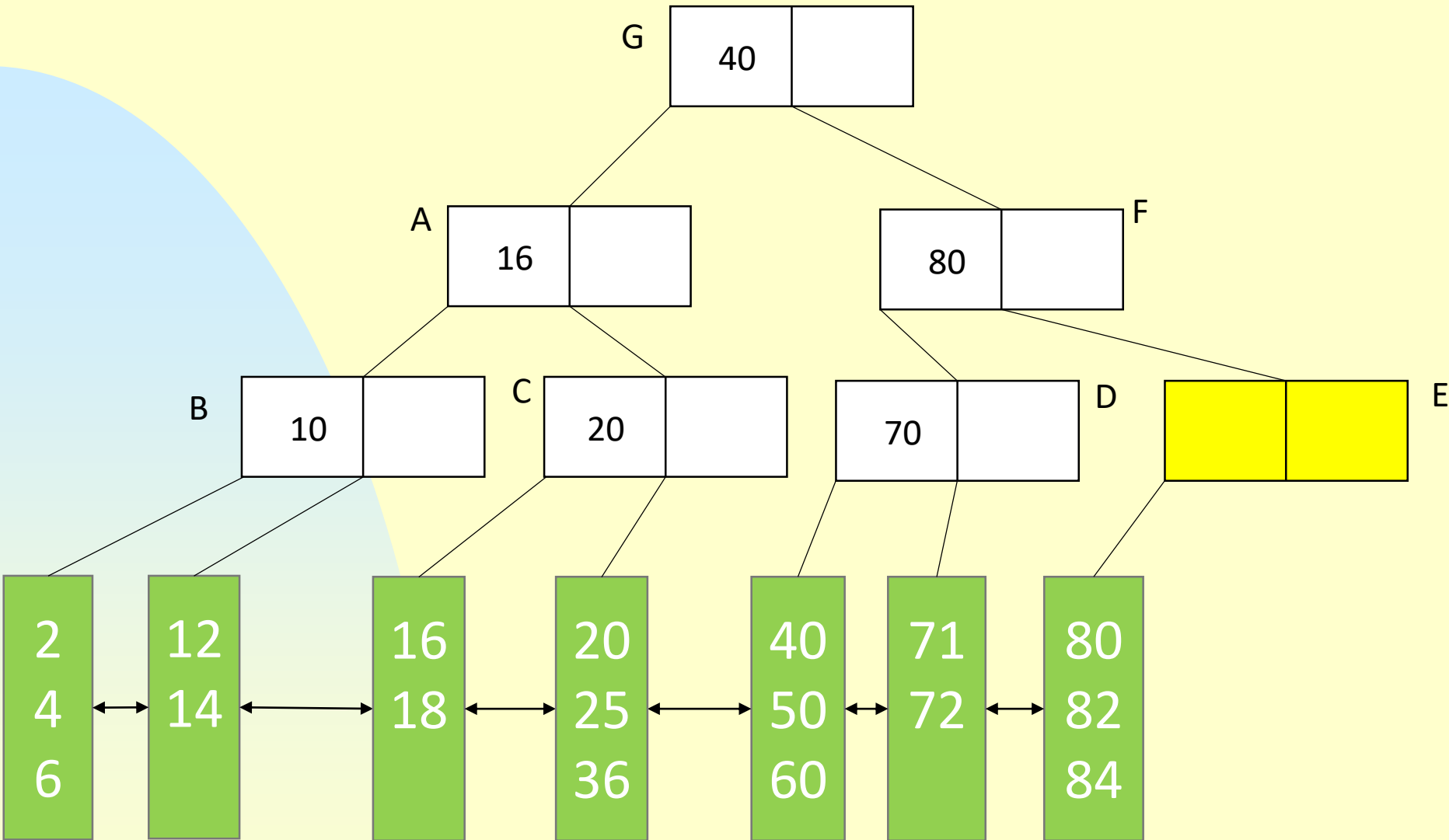
C is deficient.



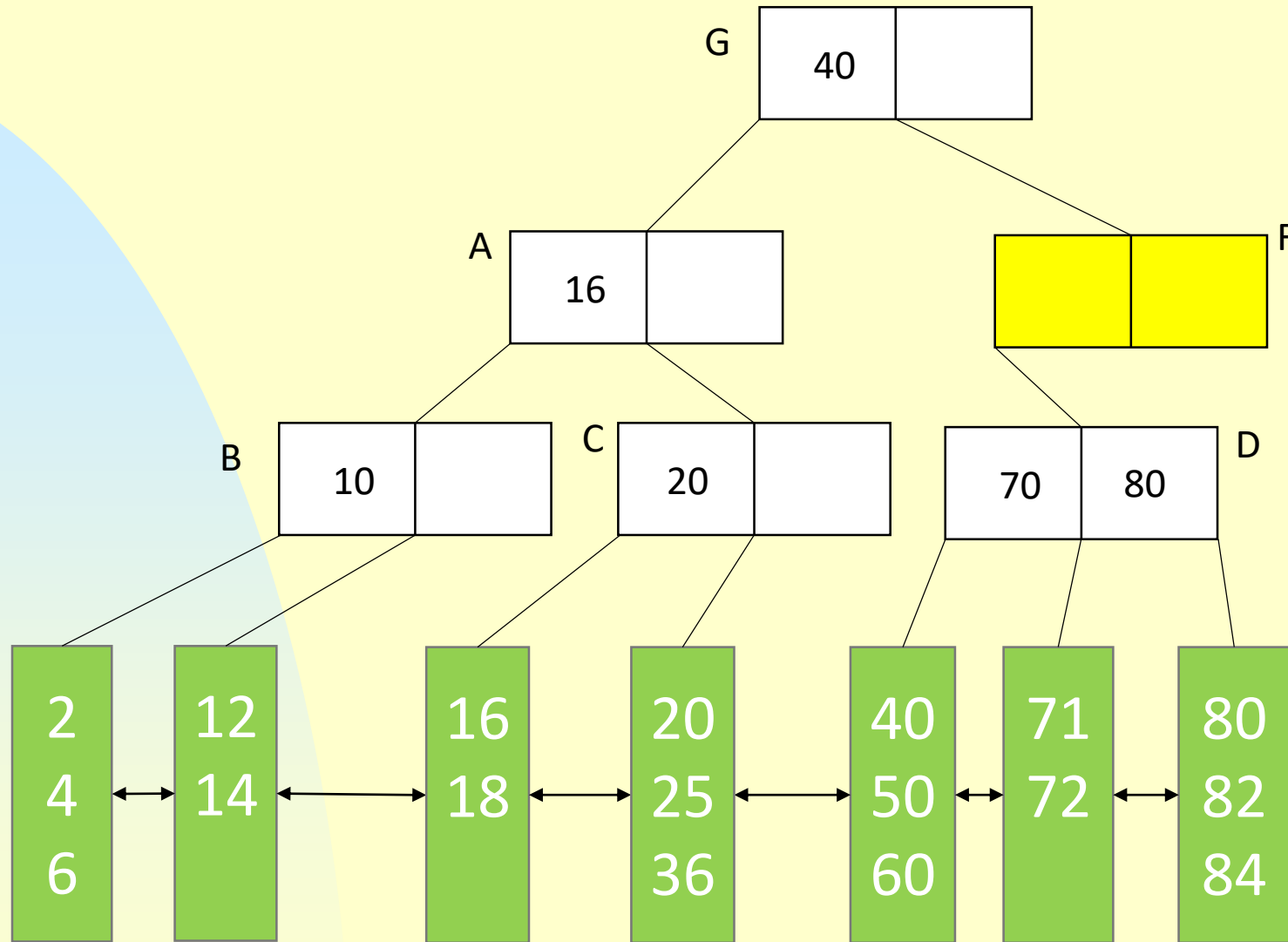
After borrowing from B.



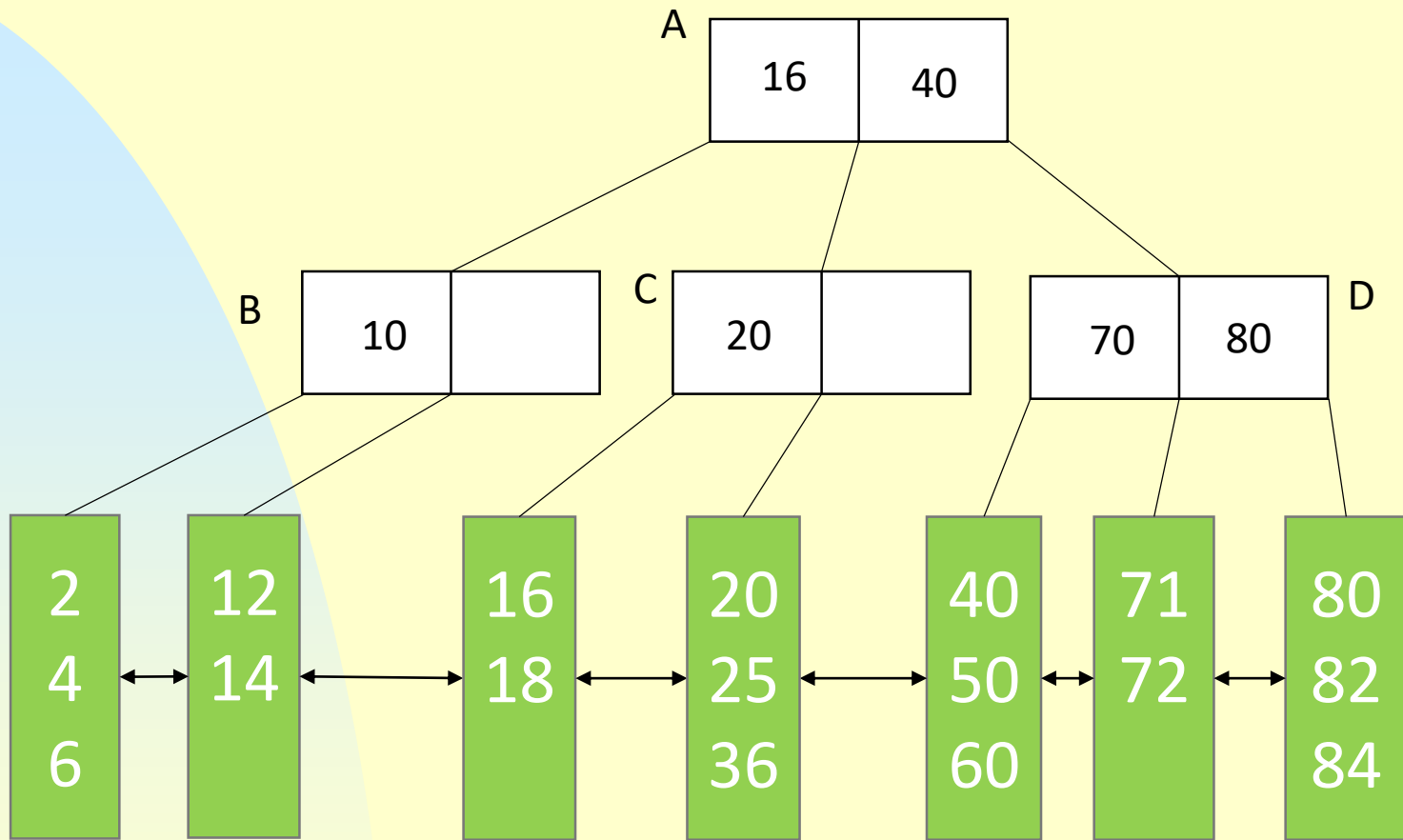
Deletion: 86



E becomes deficient.



F becomes deficient.



The final B+-tree after 86 deleted.

Exercise(s) and Experiment(s)

Exercise(s):

- **none**

Experiment(s):

- **none**